

# Local Voice Chatbot

A modulation-robust, fully offline speech-to-speech assistant integrating on-device speech recognition, a local large language model, and neural text-to-speech.

This document specifies the functional and non-functional requirements, system architecture, detailed design, verification strategy, experimental evaluation, and ethical assessment of the Local Voice Chatbot system. The system executes a complete speech-to-speech conversational pipeline entirely on local hardware, with no external network dependency at any stage. It incorporates an explicit confidence-gating layer that prevents low-confidence transcriptions from reaching the language model, and a measured, benchmark-driven approach to validating robustness claims — including a documented negative experimental result and the corrective action taken in response to it.

|                |                                                                         |
|----------------|-------------------------------------------------------------------------|
| DOCUMENT ID    | LVC-FTS-001                                                             |
| VERSION        | 1.0 (Baselined)                                                         |
| STATUS         | Released for Review                                                     |
| DATE           | 07 September 2026                                                       |
| AUTHOR         | Sandeep Srinivasan                                                      |
| CLASSIFICATION | Internal — Project Submission                                           |
| REPOSITORY     | voice-chatbot (git, branch <code>master</code> , 7 commits at baseline) |

## § Document Control

### Revision History

| Version | Date        | Author        | Summary of Changes                                                                                                                                                   |
|---------|-------------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0.1     | 06 Sep 2026 | S. Srinivasan | Initial draft: scope, architecture outline, preliminary requirements.                                                                                                |
| 0.2     | 06 Sep 2026 | S. Srinivasan | Added detailed design for preprocessing and confidence layers; incorporated GPU enablement findings.                                                                 |
| 0.3     | 07 Sep 2026 | S. Srinivasan | Added experimental evaluation following first full benchmark execution; documented negative result and root-cause analysis.                                          |
| 1.0     | 07 Sep 2026 | S. Srinivasan | Baselined. Added verification & validation, security, ethical assessment, operations, and appendices. Aligned with repository state at commit <code>383e028</code> . |

### Approvals

| Role                | Name               | Date        | Signature |
|---------------------|--------------------|-------------|-----------|
| Author / Developer  | Sandeep Srinivasan | 07 Sep 2026 |           |
| Technical Reviewer  |                    |             |           |
| Approving Authority |                    |             |           |

### Distribution

| Recipient                | Format                                 | Purpose                                         |
|--------------------------|----------------------------------------|-------------------------------------------------|
| Engineering Review Panel | PDF (this document)                    | Technical assessment and design review.         |
| Project Repository       | Markdown<br>( <code>README.md</code> ) | Condensed operational reference for developers. |
| Reviewer / Assessor      | PDF + source repository                | Evaluation of engineering practice and results. |

### Related Documents

| Reference              | Artefact               | Description                                                       |
|------------------------|------------------------|-------------------------------------------------------------------|
| <a href="#">REF-01</a> | <code>README.md</code> | Operational quick-reference: setup, run modes, benchmark summary. |

| Reference | Artefact                                          | Description                                                          |
|-----------|---------------------------------------------------|----------------------------------------------------------------------|
| REF-02    | <code>data/results/benchmark_results.csv</code>   | Per-clip raw experimental output of the evaluation described in §12. |
| REF-03    | <code>data/results/accuracy_comparison.png</code> | Rendered accuracy comparison chart (reproduced as Figure 3).         |
| REF-04    | <code>logs/confidence_log.csv</code>              | Audit trail of every confidence-gating decision taken at runtime.    |
| REF-05    | <code>.github/workflows/tests.yml</code>          | Continuous integration pipeline definition.                          |

## § Table of Contents

---

|     |                                                 |    |
|-----|-------------------------------------------------|----|
| 1   | Executive Summary .....                         | 6  |
| 2   | Introduction .....                              | 7  |
| 2.1 | Purpose .....                                   | 7  |
| 2.2 | Intended Audience .....                         | 7  |
| 2.3 | Scope .....                                     | 7  |
| 2.4 | Definitions, Acronyms and Abbreviations .....   | 7  |
| 3   | Product Overview .....                          | 9  |
| 3.1 | Problem Statement .....                         | 9  |
| 3.2 | Solution Overview .....                         | 9  |
| 3.3 | User Characteristics .....                      | 9  |
| 3.4 | Operating Environment .....                     | 9  |
| 3.5 | Constraints, Assumptions and Dependencies ..... | 9  |
| 4   | Functional Requirements .....                   | 12 |
| 5   | Non-Functional Requirements .....               | 16 |
| 6   | Use Cases .....                                 | 19 |
| 7   | System Architecture .....                       | 22 |
| 8   | Detailed Design .....                           | 26 |
| 9   | Data Design .....                               | 32 |
| 10  | Interface Specification .....                   | 34 |
| 11  | Verification & Validation .....                 | 36 |
| 12  | Experimental Evaluation .....                   | 39 |
| 13  | Performance Characteristics .....               | 45 |
| 14  | Security Considerations .....                   | 47 |
| 15  | Ethical Considerations .....                    | 49 |
| 16  | Operations & Deployment .....                   | 54 |
| 17  | Known Limitations .....                         | 57 |
| 18  | Future Work .....                               | 59 |
| 19  | Appendices .....                                | 61 |

## Figures and Tables

|       |                                                |    |
|-------|------------------------------------------------|----|
| Fig 1 | End-to-end processing pipeline .....           | 22 |
| Fig 2 | Confidence-gating decision state machine ..... | 26 |



# 1 Executive Summary

The Local Voice Chatbot is a speech-to-speech conversational assistant that runs its entire processing chain — speech recognition, language modelling, and speech synthesis — on local hardware. No audio, transcript, or generated response is transmitted to any external service at any point. The system was developed to address a specific reliability problem in voice interfaces: speech recognition models degrade when input is delivered under atypical vocal modulation (whispered, shouted, unusually slow or fast), and they typically fail *silently*, emitting a confident but incorrect transcription that downstream components then act upon.

The system addresses this in two layers. First, an audio preprocessing stage normalises input before recognition. Second — and more consequentially — a confidence-gating layer inspects the recogniser's own token-level log-probability for every utterance and refuses to forward low-confidence transcriptions to the language model, instead re-attempting recognition with a wider beam search and, failing that, explicitly asking the speaker to repeat. Every such decision is written to a persistent audit log.

|                                                                   |                                                         |                                                     |                                                       |
|-------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------|
| <b>96.0%</b><br>RECOGNITION ACCURACY<br>ACROSS ALL<br>MODULATIONS | <b>42</b><br>AUTOMATED TESTS, GPU-<br>FREE, CI-ENFORCED | <b>2.6 GB</b><br>PEAK VRAM, BOTH<br>MODELS RESIDENT | <b>0</b><br>EXTERNAL NETWORK<br>CALLS IN THE PIPELINE |
|-------------------------------------------------------------------|---------------------------------------------------------|-----------------------------------------------------|-------------------------------------------------------|

## Principal Outcomes

- **A complete, working offline pipeline.** Microphone capture with voice-activity detection, GPU-accelerated recognition via `faster-whisper`, response generation via a locally-served 3-billion-parameter language model, and neural speech synthesis via Piper — all verified operating end-to-end on a 4 GB consumer laptop GPU.
- **A confidence-aware safety layer** that converts silent recognition failures into explicit, recoverable interactions, with a complete audit trail.
- **A documented negative experimental result.** The pitch- and speed-correction components of the preprocessing layer were hypothesised to improve robustness. Benchmarking against a purpose-recorded 25-clip dataset demonstrated the opposite: they reduced accuracy from 96.0% to 48.0%. The cause was isolated through controlled ablation, the components were disabled, accuracy was restored to parity, and an automated regression test now prevents the defect from recurring. This result, and the decision taken in response to it, is reported in full in §12.
- **Production-oriented engineering practice:** centralised and validated configuration, a 42-test automated suite with continuous integration, consistent structured error handling, retry semantics on inter-process calls, log rotation, and a production WSGI server option — with the deliberately unimplemented items enumerated honestly in §17.

### PRINCIPAL ENGINEERING FINDING

The measured value of this project lies less in the pipeline itself — which is an integration of well-understood components — than in the evidence discipline applied to it. A plausible, intuitively-correct preprocessing technique was implemented, measured, found to be actively harmful, root-caused, removed, and regression-tested. The benchmark existed specifically so that such a claim could be falsified rather than assumed.

## 2 Introduction

### 2.1 Purpose

This document defines, in a single controlled artefact, what the Local Voice Chatbot system does, how it is constructed, how its behaviour has been verified, and what its residual risks and limitations are. It serves three functions: as a *specification* against which the delivered implementation can be assessed; as a *design record* explaining why each significant technical decision was taken; and as an *evidence dossier* presenting the measured results that support or, in one important case, refute the system's design hypotheses.

### 2.2 Intended Audience

| Audience                        | Primary sections           | Reading objective                                                       |
|---------------------------------|----------------------------|-------------------------------------------------------------------------|
| Technical reviewer / assessor   | \$1, \$7, \$11, \$12, \$15 | Assess engineering quality, evidence discipline, and ethical reasoning. |
| Engineer maintaining the system | \$7, \$8, \$9, \$10, \$16  | Understand the architecture and modify it safely.                       |
| Reviewer of requirements        | \$3, \$4, \$5, \$6, \$11.6 | Confirm requirement completeness and verification coverage.             |
| Operator / end user             | \$16, \$17                 | Install, configure, run, and troubleshoot the system.                   |

### 2.3 Scope

#### 2.3.1 In Scope

- Real-time speech capture with automatic endpoint detection, on a single local machine.
- Offline speech-to-text with GPU acceleration and automatic CPU fallback.
- Confidence assessment of each transcription, with retry and explicit-rejection behaviour.
- Response generation by a locally-hosted large language model, with multi-turn context.
- Offline neural speech synthesis and playback of the generated response.
- Two operator interfaces: a terminal client and a browser-based chat interface.
- Tooling to record a personal evaluation dataset and to execute a reproducible accuracy benchmark.
- Configuration management, automated testing, continuous integration, logging, and error handling.

#### 2.3.2 Out of Scope

- **Authentication and authorisation.** The system is designed for single-user operation on `127.0.0.1`. It has no user accounts, sessions, or access control.
- **Multi-user or concurrent operation.** One recogniser instance, no request queueing, no tenancy isolation.

- **Internet-facing deployment.** Hosting the system publicly would contradict its core design premise (\$15.1) and is explicitly not supported.
- **Packaging and distribution** (installer, container image, signed release).
- **Wake-word activation, barge-in interruption, and speaker identification.**
- **Languages other than English.** Recognition is pinned to English; the synthesis voice is US English.

## 2.4 Definitions, Acronyms and Abbreviations

| Term            | Definition                                                                                                                              |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| STT             | Speech-to-Text. Conversion of an audio waveform into a text transcription. Also "ASR" (Automatic Speech Recognition).                   |
| TTS             | Text-to-Speech. Synthesis of an audio waveform from text.                                                                               |
| LLM             | Large Language Model. Here, a 3.2-billion-parameter instruction-tuned transformer served locally.                                       |
| VAD             | Voice Activity Detection. Frame-level classification of audio as speech or non-speech; used to determine when the speaker has finished. |
| avg_logprob     | The mean log-probability the recogniser assigned to the tokens it emitted for a segment. Used here as an uncalibrated confidence proxy. |
| Modulation      | In this document, a deliberate variation in vocal delivery of the same word — normal, whispered, shouted, slow, or fast.                |
| Confidence gate | The decision layer that accepts, re-attempts, or rejects a transcription based on its <code>avg_logprob</code> .                        |
| Beam size       | Width of the decoder's search. A larger beam explores more candidate token sequences at higher computational cost.                      |
| RMS             | Root Mean Square. Used as an amplitude measure that reflects sustained loudness rather than peak level.                                 |
| f0              | Fundamental frequency — the perceived pitch of voiced speech.                                                                           |
| WSGI            | Web Server Gateway Interface. The Python standard interface between web servers and applications.                                       |
| Ablation        | An experimental technique in which components are selectively disabled to attribute an observed effect to a specific cause.             |

### 3 Product Overview

#### 3.1 Problem Statement

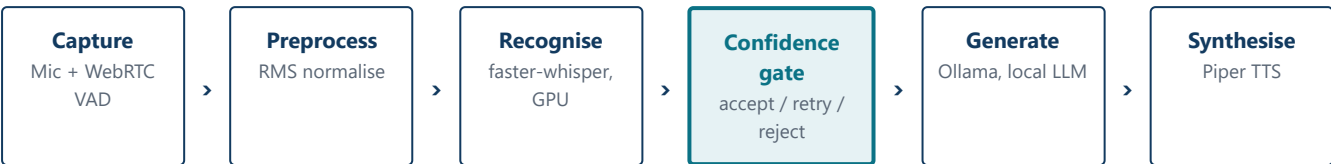
Speech recognition models are trained predominantly on conversational and read speech delivered at typical loudness, pitch, and rate. When a speaker departs from that distribution — whispering in a quiet room, raising their voice across a distance, deliberately slowing down for emphasis, or rushing — recognition accuracy falls. The operational hazard is not the accuracy loss itself but its *failure mode*: the model does not signal uncertainty. It emits a fluent, plausible, and incorrect transcription, which any downstream component will then treat as ground truth.

In a voice assistant this produces a characteristic and damaging user experience: the assistant answers a question the user did not ask, with no indication that it misheard. For command words in particular — `stop`, `help` — a silent substitution is a functional failure, not merely an annoyance.

A second constraint shapes the design. Voice data is among the most sensitive categories of personal data: it carries identity, emotional state, health signals, and whatever the speaker happens to say in their own home. Any architecture that transmits raw audio to a third-party service inherits an irreducible privacy exposure. This project therefore adopts a strict constraint: nothing leaves the machine.

#### 3.2 Solution Overview

The system implements a five-stage pipeline in which every stage executes locally, together with a decision gate between recognition and generation.



**Figure 1.** End-to-end processing pipeline. Every stage executes on local hardware; the only network traffic is loopback to `127.0.0.1`. The confidence gate is the sole point at which the pipeline may decline to proceed and return control to the speaker.

The confidence gate is the architecturally significant element. Rather than forcing the recogniser to always produce an answer, the system treats "I am not sufficiently sure" as a first-class outcome: it re-decodes with a wider beam, and if confidence remains below threshold it asks the speaker to repeat rather than guessing. This mirrors how uncertainty is handled in production speech systems and converts a silent failure into a recoverable interaction.

#### 3.3 User Characteristics

| User class                  | Technical proficiency                  | Interaction with the system                                                                                              |
|-----------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Primary operator (end user) | Low — no programming knowledge assumed | Speaks to the assistant through the browser interface; reads transcriptions and responses; may type instead of speaking. |

| User class             | Technical proficiency                   | Interaction with the system                                                             |
|------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------|
| Developer / maintainer | High — Python, CLI, basic ML operations | Installs dependencies, configures the system, runs the test suite, extends modules.     |
| Evaluator              | Moderate to high                        | Records an evaluation dataset, executes the benchmark, inspects results and audit logs. |

### 3.4 Operating Environment

The system was developed and verified on the following reference configuration. It is not restricted to this configuration, but performance characteristics in §13 are stated against it.

| Element                | Reference configuration (as verified)                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------------|
| Operating system       | Windows 11 (build 10.0.26200)                                                                                |
| Python runtime         | CPython 3.11.9, isolated virtual environment                                                                 |
| GPU                    | NVIDIA GeForce RTX 3050 Laptop GPU, 4096 MiB VRAM, driver 581.95                                             |
| System memory          | 16 GB                                                                                                        |
| Recognition engine     | faster-whisper 1.2.1 on CTranslate2 4.8.2; model <code>small</code> , compute type <code>int8_float16</code> |
| Language model runtime | Ollama 0.33.3 serving <code>llama3.2:3b</code> (3.2 B parameters, Q4_K_M quantisation, 2.0 GB)               |
| Synthesis engine       | piper-tts 1.8.0, voice <code>en_US-lessac-medium</code> (60 MB ONNX), 22.05 kHz mono output                  |
| Browser                | Any Chromium- or Gecko-based browser supporting <code>MediaRecorder</code> and <code>getUserMedia</code>     |

### 3.5 Constraints, Assumptions and Dependencies

| ID            | Type       | Statement and consequence                                                                                                                                                                      |
|---------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CON-01</b> | Constraint | <b>No external network calls.</b> Every component must operate against local files or loopback addresses. This eliminates cloud STT/LLM/TTS options entirely and drives the selection in §7.4. |
| <b>CON-02</b> | Constraint | <b>4 GB VRAM budget, shared.</b> The recogniser and the language model must be simultaneously resident. This bounds model sizes and mandates quantisation (§13.2).                             |
| <b>CON-03</b> | Constraint | <b>Consumer Windows host.</b> Precludes deployment tooling that assumes Linux (e.g. <code>gunicorn</code> ), and introduces the CUDA library resolution problem documented in §8.4.2.          |
| <b>CON-04</b> | Constraint | <b>Browser microphone access requires a secure context.</b> <code>getUserMedia</code> is available only over HTTPS or on <code>localhost</code> ; the latter is used.                          |
| <b>ASM-01</b> | Assumption | A single speaker uses the system at a time, in a reasonably quiet environment.                                                                                                                 |

| ID     | Type       | Statement and consequence                                                                                                                                                                                        |
|--------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ASM-02 | Assumption | The operator has administrative rights sufficient to install the Ollama runtime.                                                                                                                                 |
| ASM-03 | Assumption | Utterances are short — command words to single sentences. This assumption is material: §12.4 shows a design defect that arose precisely from applying a technique valid for continuous speech to isolated words. |
| DEP-01 | Dependency | Ollama must be installed and running, with the configured model pulled, before the pipeline can generate responses.                                                                                              |
| DEP-02 | Dependency | A Piper voice model ( <code>.onnx</code> plus <code>.onnx.json</code> ) must be present at the configured path.                                                                                                  |
| DEP-03 | Dependency | GPU acceleration requires the cuBLAS and cuDNN runtime libraries; absent these, the system degrades automatically to CPU (§8.4).                                                                                 |

## 4 Functional Requirements

Requirements are identified as `FR-<group>.<n>`. Priority follows MoSCoW convention. *Status* reflects the implementation state at document baseline. *Verification* identifies the method by which conformance is demonstrated; the full traceability matrix is given in §11.6.

### 4.1 Audio Capture

| ID     | Requirement                                                                                                                               | Priority | Status | Verification  |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------|
| FR-1.1 | The system shall capture audio from the host's default input device at 16 kHz, mono, 16-bit.                                              | MUST     | DONE   | Demonstration |
| FR-1.2 | The system shall detect the end of an utterance automatically using voice-activity detection, rather than recording for a fixed duration. | MUST     | DONE   | Demonstration |
| FR-1.3 | The system shall retain a pre-trigger audio buffer ( $\approx 300$ ms) so that speech onset is not clipped.                               | SHOULD   | DONE   | Inspection    |
| FR-1.4 | The system shall discard candidate utterances shorter than 150 ms of detected speech as spurious.                                         | SHOULD   | DONE   | Inspection    |
| FR-1.5 | The system shall terminate capture after a configurable maximum duration (default 15 s) irrespective of VAD state.                        | MUST     | DONE   | Inspection    |
| FR-1.6 | The browser interface shall permit the user to start and stop recording explicitly, without a fixed time limit.                           | MUST     | DONE   | Demonstration |

### 4.2 Audio Preprocessing

| ID     | Requirement                                                                                                                                                                   | Priority | Status | Verification    |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|-----------------|
| FR-2.1 | The system shall normalise input amplitude to a consistent RMS level prior to recognition.                                                                                    | MUST     | DONE   | Unit test       |
| FR-2.2 | Amplitude normalisation shall not introduce clipping when applying large gain to quiet input.                                                                                 | MUST     | DONE   | Unit test       |
| FR-2.3 | The system shall detect and record the median fundamental frequency of voiced input, flagging values outside 80–400 Hz as outliers.                                           | SHOULD   | DONE   | Unit test       |
| FR-2.4 | The system shall estimate and record speaking rate, flagging values outside the normal band.                                                                                  | SHOULD   | DONE   | Unit test       |
| FR-2.5 | Pitch and rate <i>correction</i> shall be implemented but disabled by default, following the evaluation in §12; enabling them shall require an explicit configuration change. | MUST     | DONE   | Regression test |

| ID     | Requirement                                                                                                                                      | Priority | Status | Verification |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|--------------|
| FR-2.6 | Any failure within preprocessing shall be logged and the unmodified audio passed through; preprocessing shall never abort a conversational turn. | MUST     | DONE   | Unit test    |

### 4.3 Speech Recognition

| ID     | Requirement                                                                                                                                                           | Priority | Status | Verification  |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------|
| FR-3.1 | The system shall transcribe captured audio using a locally-stored recognition model, with no network access.                                                          | MUST     | DONE   | Demonstration |
| FR-3.2 | The system shall use GPU acceleration when available.                                                                                                                 | MUST     | DONE   | Demonstration |
| FR-3.3 | The system shall verify GPU usability by executing a trial inference at load time, not by construction success alone, and shall fall back to CPU if that trial fails. | MUST     | DONE   | Demonstration |
| FR-3.4 | The system shall expose the per-segment confidence score of every transcription to the calling layer.                                                                 | MUST     | DONE   | Unit test     |
| FR-3.5 | The recogniser shall load once per process and be reused across turns.                                                                                                | SHOULD   | DONE   | Inspection    |

### 4.4 Confidence Gating

| ID     | Requirement                                                                                                                                                                           | Priority | Status | Verification     |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|------------------|
| FR-4.1 | The system shall compute a single confidence value per utterance as the duration-weighted mean of segment log-probabilities.                                                          | MUST     | DONE   | Unit test        |
| FR-4.2 | An utterance yielding no decoded segments shall be treated as a definitive rejection.                                                                                                 | MUST     | DONE   | Unit test        |
| FR-4.3 | Where confidence is at or above the configured threshold, the transcription shall be accepted and forwarded.                                                                          | MUST     | DONE   | Unit test        |
| FR-4.4 | Where confidence is below threshold on the first attempt, the system shall re-transcribe with an increased beam size before deciding.                                                 | MUST     | DONE   | Unit test        |
| FR-4.5 | Where confidence remains below threshold after re-transcription, the system shall not forward the text to the language model, and shall instead request repetition audibly.           | MUST     | DONE   | Unit test + demo |
| FR-4.6 | Every gating decision — accepted and rejected alike — shall be appended to a persistent CSV audit log with timestamp, text, score, threshold, decision, beam size and attempt number. | MUST     | DONE   | Unit test        |

| ID     | Requirement                                                                                                                                                | Priority | Status | Verification  |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------|
| FR-4.7 | The browser interface shall visually distinguish a rejected transcription from an accepted one and display the confidence score that caused the rejection. | SHOULD   | DONE   | Demonstration |

## 4.5 Response Generation

| ID     | Requirement                                                                                                                               | Priority | Status | Verification     |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|------------------|
| FR-5.1 | The system shall generate responses using a language model served on the local loopback interface.                                        | MUST     | DONE   | Demonstration    |
| FR-5.2 | Responses shall be streamed incrementally rather than returned only on completion.                                                        | SHOULD   | DONE   | Integration test |
| FR-5.3 | The system shall constrain response style, by system prompt, to short spoken-form prose without lists, headings, or markup.               | MUST     | DONE   | Demonstration    |
| FR-5.4 | The browser interface shall maintain multi-turn conversational context across a session.                                                  | SHOULD   | DONE   | Integration test |
| FR-5.5 | Transient connection failures to the model server shall be retried with backoff, provided no output has yet been delivered to the caller. | SHOULD   | DONE   | Integration test |
| FR-5.6 | Where the model server is unreachable, the system shall present a clear diagnostic naming the endpoint and the corrective action.         | MUST     | DONE   | Integration test |

## 4.6 Speech Synthesis and Interfaces

| ID     | Requirement                                                                                                                   | Priority | Status | Verification  |
|--------|-------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------|
| FR-6.1 | The system shall synthesise the generated response to speech using a local engine and voice model.                            | MUST     | DONE   | Demonstration |
| FR-6.2 | Synthesised audio shall be played back automatically on completion of generation.                                             | MUST     | DONE   | Demonstration |
| FR-6.3 | The browser interface shall allow any previous assistant response to be replayed without regenerating it.                     | COULD    | DONE   | Demonstration |
| FR-7.1 | The system shall provide a terminal interface presenting listening state, transcription, confidence, and response.            | MUST     | DONE   | Demonstration |
| FR-7.2 | The system shall provide a browser interface with conversational message history, microphone control, and streamed responses. | MUST     | DONE   | Demonstration |
| FR-7.3 | The browser interface shall accept typed input as an alternative to speech.                                                   | SHOULD   | DONE   | Demonstration |

| ID     | Requirement                                                                                                                                                 | Priority | Status | Verification  |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------|
| FR-7.4 | Both interfaces shall perform preflight checks (microphone, model server, voice model) and report failures as actionable messages rather than stack traces. | MUST     | DONE   | Demonstration |

## 4.7 Evaluation Tooling and Operations

| ID     | Requirement                                                                                                                                                                                        | Priority | Status | Verification  |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------|---------------|
| FR-8.1 | The system shall provide tooling to record a labelled evaluation dataset spanning defined words and modulations.                                                                                   | MUST     | DONE   | Demonstration |
| FR-8.2 | Recording tooling shall permit playback and re-take of each clip before it is committed.                                                                                                           | SHOULD   | DONE   | Demonstration |
| FR-8.3 | Recording tooling shall resume from the last completed clip if interrupted.                                                                                                                        | COULD    | DONE   | Demonstration |
| FR-8.4 | The system shall provide a benchmark that transcribes each dataset clip both with and without preprocessing under otherwise identical conditions, and reports accuracy overall and per modulation. | MUST     | DONE   | Demonstration |
| FR-8.5 | Benchmark output shall be persisted as machine-readable results and a rendered chart.                                                                                                              | MUST     | DONE   | Inspection    |
| FR-9.1 | All configurable parameters shall be defined in a single validated module, overridable by environment variable or <code>.env</code> file.                                                          | MUST     | DONE   | Unit test     |
| FR-9.2 | Invalid configuration values shall raise a diagnostic naming the offending variable.                                                                                                               | SHOULD   | DONE   | Unit test     |
| FR-9.3 | The system shall provide an installation verification utility reporting the status of each external dependency independently.                                                                      | MUST     | DONE   | Demonstration |
| FR-9.4 | Per-stage timings shall be recorded to a rotating log for latency analysis.                                                                                                                        | SHOULD   | DONE   | Inspection    |

## 5 Non-Functional Requirements

Non-functional requirements are identified as `NFR-<group>.<n>`. Where a requirement is quantified, the value measured on the reference configuration is stated here and detailed in §13.

### 5.1 Performance

| ID                      | Requirement                                                                                                                                                            | Target / measured                               |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <a href="#">NFR-P.1</a> | Speech recognition of a short utterance shall complete within 2 s on the reference GPU configuration.                                                                  | Target < 2.0 s;<br><b>measured 0.85 s</b>       |
| <a href="#">NFR-P.2</a> | The recogniser and language model shall be simultaneously resident within the 4 GB VRAM budget, retaining headroom for display and compositor allocations.             | Target < 3.5 GB;<br><b>measured 2.6 GB peak</b> |
| <a href="#">NFR-P.3</a> | The language model shall begin streaming output before generation completes, so perceived latency is bounded by time-to-first-token rather than total generation time. | Implemented                                     |
| <a href="#">NFR-P.4</a> | The recognition model shall load once per process and be reused across turns.                                                                                          | Implemented (lazy, cached)                      |
| <a href="#">NFR-P.5</a> | First-turn cold-start latency may exceed steady-state latency, but shall be communicated to the user rather than presenting as an unexplained delay.                   | Implemented (status text)                       |

### 5.2 Reliability and Robustness

| ID                      | Requirement                                                                                                                                          | Realisation                            |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| <a href="#">NFR-R.1</a> | No single non-critical subsystem failure shall terminate a conversational session.                                                                   | Fail-closed handlers per stage (§8.9)  |
| <a href="#">NFR-R.2</a> | Failure of audio preprocessing shall degrade to unprocessed audio, not to an aborted turn.                                                           | Two-level guard: per-step and per-turn |
| <a href="#">NFR-R.3</a> | Absence or malfunction of GPU acceleration shall degrade to CPU execution automatically, without operator intervention.                              | Trial-inference probe at load (§8.4)   |
| <a href="#">NFR-R.4</a> | Transient failures of the language model server shall be retried before being surfaced as errors, but only where no partial output has been emitted. | Bounded retry with backoff (§8.6.3)    |
| <a href="#">NFR-R.5</a> | Log files shall not grow without bound.                                                                                                              | Rotating handler, 5 MB × 3 generations |
| <a href="#">NFR-R.6</a> | Evaluation dataset recording shall be resumable after interruption.                                                                                  | Manifest-driven resume (§8.8.3)        |

## 5.3 Security and Privacy

| ID      | Requirement                                                                                                            | Realisation                                               |
|---------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| NFR-S.1 | No component shall transmit audio, transcripts, or generated text to any host other than the local loopback interface. | Architectural; verified by network isolation test (§11.5) |
| NFR-S.2 | Web services shall bind exclusively to <code>127.0.0.1</code> and shall not listen on externally routable interfaces.  | Explicit bind address                                     |
| NFR-S.3 | Uploaded audio shall be size-bounded to prevent unbounded resource consumption.                                        | 25 MB cap; HTTP 413 on breach                             |
| NFR-S.4 | Error responses shall not disclose stack traces or internal filesystem paths to the client.                            | Structured JSON error handlers (§8.8.1)                   |
| NFR-S.5 | Recorded voice data shall remain on the local filesystem and shall be excluded from version control.                   | <code>.gitignore</code> excludes <code>*.wav</code>       |

## 5.4 Usability

| ID      | Requirement                                                                                                        | Realisation                          |
|---------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------|
| NFR-U.1 | Dependency failures shall be reported as actionable prose naming the remedy, not as stack traces.                  | Preflight checks in both interfaces  |
| NFR-U.2 | The user shall be informed of the current activity — listening, transcribing, generating, speaking — at all times. | Live status line                     |
| NFR-U.3 | Rejected transcriptions shall be presented with the reason for rejection, not silently discarded.                  | Distinct visual treatment plus score |
| NFR-U.4 | Generated responses shall be phrased for auditory delivery, avoiding constructs that depend on visual layout.      | System prompt constraint (§8.6.1)    |
| NFR-U.5 | A non-vocal input path shall exist for environments where speech is impractical.                                   | Text entry in browser interface      |

## 5.5 Maintainability and Portability

| ID      | Requirement                                                                                                      | Realisation                                   |
|---------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| NFR-M.1 | All tunable parameters shall be defined once, in a single module, and shall not be duplicated across call sites. | <code>src/config.py</code> , frozen dataclass |
| NFR-M.2 | The automated test suite shall execute without GPU, language model server, microphone, or synthesis binary.      | 42 tests, all external calls mocked           |
| NFR-M.3 | Tests and static analysis shall execute automatically on every change pushed to the repository.                  | GitHub Actions workflow                       |

| ID             | Requirement                                                                                                                          | Realisation                           |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| <b>NFR-M.4</b> | The codebase shall pass static lint analysis with no outstanding findings.                                                           | <code>ruff</code> ; clean at baseline |
| <b>NFR-M.5</b> | Behaviour established by empirical evidence shall be protected by regression tests that fail if it is reverted without new evidence. | Preprocessing lock-in tests (§11.3)   |
| <b>NFR-M.6</b> | Dependencies shall be layered so environments not requiring GPU acceleration need not install GPU runtime libraries.                 | Three-file requirements split         |
| <b>NFR-M.7</b> | Platform-specific workarounds shall be isolated in dedicated modules and documented at the point of use.                             | <code>src/cuda_dlls.py</code>         |

## 5.6 Observability

| ID             | Requirement                                                                                               | Realisation                                           |
|----------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>NFR-O.1</b> | Per-stage timing shall be recorded for every conversational turn.                                         | <code>StageTimings</code> → <code>pipeline.log</code> |
| <b>NFR-O.2</b> | Every confidence decision shall be independently auditable after the fact, including decisions to accept. | <code>confidence_log.csv</code>                       |
| <b>NFR-O.3</b> | The active compute device for recognition shall be discoverable at runtime by the operator.               | Health endpoint and interface badge                   |

# 6 Use Cases

## 6.1 Actor Inventory

| Actor          | Type               | Role                                                                        |
|----------------|--------------------|-----------------------------------------------------------------------------|
| Speaker / User | Primary, human     | Initiates conversational turns by voice or text; consumes spoken responses. |
| Evaluator      | Secondary, human   | Records the evaluation dataset and executes benchmarks.                     |
| Ollama service | Supporting, system | Serves the language model over the loopback interface.                      |
| Piper binary   | Supporting, system | Performs speech synthesis as a subprocess.                                  |

## 6.2 UC-01 — Conduct a spoken conversational turn

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identifier     | UC-01                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| Primary actor  | Speaker                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| Goal           | Obtain a spoken answer to a spoken question, with no data leaving the machine.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Preconditions  | Service running; microphone permission granted; Ollama reachable; voice model present.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Trigger        | User activates the microphone control and speaks.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| Main flow      | <div>1. User starts recording; the interface indicates the listening state.</div> <div>2. User speaks, then stops recording.</div> <div>3. System decodes the browser audio to 16 kHz mono PCM.</div> <div>4. System normalises amplitude (§8.3).</div> <div>5. System transcribes the audio and computes a confidence score.</div> <div>6. The confidence gate accepts the transcription (§8.5).</div> <div>7. The transcription is displayed as the user message, annotated with its confidence.</div> <div>8. System submits the transcription, prior turns, and system prompt to the language model.</div> <div>9. Response tokens stream into the interface as they are produced.</div> <div>10. On completion, the response is synthesised and played automatically.</div> <div>11. The turn is appended to session history; the timing record is written.</div> |
| Postconditions | Conversation history extended by one exchange; one row appended to the confidence audit log; per-stage timings recorded.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternate flows | <p><b>A1 — Low confidence.</b> At step 6 the gate rejects; control transfers to UC-02.</p> <p><b>A2 — No speech detected.</b> Capture yields insufficient voiced audio; the turn is abandoned and the system returns to listening.</p> <p><b>A3 — Model server unreachable.</b> At step 8, after bounded retries, a diagnostic replaces the response.</p> <p><b>A4 — Synthesis failure.</b> At step 10 the textual response remains displayed; playback is skipped and an advisory is shown.</p> |
|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

### 6.3 UC-02 — Recover from a low-confidence transcription

|                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Identifier     | UC-02                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Primary actor  | Speaker                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| Goal           | Prevent an unreliable transcription from being acted upon, and give the speaker an explicit opportunity to repeat.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Preconditions  | An utterance has been transcribed with confidence below the configured threshold.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| Main flow      | <ol style="list-style-type: none"> <li>1. The gate evaluates the first-pass transcription as below threshold and returns <code>RETRY_MORE_BEAMS</code>.</li> <li>2. System re-transcribes the same audio at beam size 5 rather than 1.</li> <li>3. The gate re-evaluates the second attempt.</li> <li>4. If now at or above threshold, the transcription is accepted and UC-01 resumes at step 7.</li> <li>5. Otherwise the gate returns <code>ASK_TO_REPEAT</code>; the transcription is <i>not</i> forwarded to the language model.</li> <li>6. The interface displays the rejected text in a visually distinct form together with its score.</li> <li>7. The system synthesises and plays a request for repetition.</li> <li>8. Control returns to the speaker.</li> </ol> |
| Postconditions | No language model invocation occurred. Two rows are appended to the audit log — one per attempt — recording both scores, both beam sizes, and both decisions.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| Notes          | This is the principal safety property of the system. Correct operation was observed during system testing, including a case in which a non-speech input produced a spurious transcription at confidence $-0.88$ and was correctly refused (§11.5).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

### 6.4 Supporting Use Cases

| ID    | Use case                  | Summary                                                                                                                                                                                                                                                               |
|-------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UC-03 | Converse by typed input   | The user types rather than speaking. Recognition and confidence gating are bypassed; generation and synthesis proceed unchanged. Provides an accessible path where speech is impractical or no microphone is available.                                               |
| UC-04 | Record evaluation dataset | The evaluator is prompted for each word and modulation pair in turn, records each clip, reviews it, and either accepts or re-takes it. Accepted clips are written as 16 kHz mono WAV with a manifest entry. Interrupted sessions resume at the first incomplete pair. |

| ID    | Use case                   | Summary                                                                                                                                                                                                             |
|-------|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| UC-05 | Execute accuracy benchmark | Every manifest clip is transcribed twice — unprocessed and preprocessed — under identical decoding parameters. Accuracy is computed overall and per modulation, written to CSV, and rendered as a comparison chart. |
| UC-06 | Verify installation        | Each external dependency (CUDA visibility, recogniser, model server, synthesiser) is exercised independently and reported pass or fail, so one broken component does not mask the status of the others.             |

## 7 System Architecture

### 7.1 Architectural Overview

The system is a layered monolith comprising a shared domain core and two interchangeable presentation layers. The core ( `pipeline.py` and its collaborators) owns all domain behaviour; the terminal and browser front ends are thin adapters over it. This is deliberate: it guarantees that both interfaces exhibit identical recognition, gating, and generation semantics, because they invoke the same functions rather than parallel implementations of the same idea.

TABLE 7.1 — LAYER RESPONSIBILITIES

| Layer                  | Modules                                                                                                                           | Responsibility                                                                                 |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Presentation           | <code>cli.py</code> ; <code>chat_web.py</code> + <code>web_chat/</code> ; <code>record_web.py</code> + <code>web_recorder/</code> | User interaction, request validation, presentation of state. Contains no domain logic.         |
| Domain / orchestration | <code>pipeline.py</code>                                                                                                          | Capture, transcription with confidence policy, generation, synthesis, turn sequencing, timing. |
| Domain services        | <code>preprocessing.py</code> ; <code>confidence_check.py</code>                                                                  | Signal conditioning; confidence evaluation, decision, and audit.                               |
| Platform / support     | <code>config.py</code> ; <code>cuda_dlls.py</code> ; <code>browser_audio.py</code> ; <code>web_common.py</code>                   | Configuration, platform workarounds, media decoding, shared web concerns.                      |
| Evaluation             | <code>benchmark.py</code> ; <code>record_test_samples.py</code> ; <code>verify_setup.py</code>                                    | Dataset construction, measurement, installation validation.                                    |

### 7.2 Component Inventory

TABLE 7.2 — MODULE INVENTORY AND PRINCIPAL DEPENDENCIES

| Module                               | Responsibility                                                                                                        | Depends on                                |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>src/config.py</code>           | Single source of truth for every tunable parameter; environment and <code>.env</code> resolution; type validation.    | <code>python-dotenv</code>                |
| <code>src/cuda_dlls.py</code>        | Places CUDA runtime library directories on the process search path before the inference backend is imported (§8.4.2). | —                                         |
| <code>src/preprocessing.py</code>    | RMS normalisation; pitch and rate detection; correction stages (disabled by default).                                 | <code>librosa</code> , <code>numpy</code> |
| <code>src/confidence_check.py</code> | Confidence aggregation, three-way decision, CSV audit trail.                                                          | <code>config</code>                       |

| Module                            | Responsibility                                                                                       | Depends on                                                                                                                            |
|-----------------------------------|------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/pipeline.py</code>      | Orchestration core: capture, recognition with retry policy, generation, synthesis, playback, timing. | <code>faster-whisper</code> ,<br><code>sounddevice</code> ,<br><code>webrtcvad</code> ,<br><code>requests</code> , domain<br>services |
| <code>src/browser_audio.py</code> | Decodes browser-supplied WebM/Opus or Ogg/Opus into 16 kHz mono float PCM.                           | <code>av</code> (PyAV)                                                                                                                |
| <code>src/web_common.py</code>    | Shared JSON error handlers; development and production serving switch.                               | <code>flask</code> , <code>waitress</code> ,<br><code>config</code>                                                                   |
| <code>src/chat_web.py</code>      | HTTP surface for the conversational interface: transcription, streamed generation, synthesis.        | <code>pipeline</code> ,<br><code>web_common</code> ,<br><code>browser_audio</code>                                                    |
| <code>src/record_web.py</code>    | HTTP surface for dataset construction: plan, upload, manifest maintenance.                           | <code>web_common</code> ,<br><code>browser_audio</code> ,<br><code>soundfile</code>                                                   |
| <code>src/cli.py</code>           | Terminal presentation with preflight dependency checks.                                              | <code>pipeline</code> , <code>rich</code>                                                                                             |
| <code>src/benchmark.py</code>     | Controlled two-condition accuracy measurement and chart rendering.                                   | <code>faster-whisper</code> ,<br><code>preprocessing</code> ,<br><code>matplotlib</code>                                              |
| <code>verify_setup.py</code>      | Independent validation of each external dependency.                                                  | All external<br>components                                                                                                            |

## 7.3 Runtime View — a single conversational turn

TABLE 7.3 — SEQUENCE OF A VOICE TURN THROUGH THE BROWSER INTERFACE

| # | Participant                   | Action                                                                        |
|---|-------------------------------|-------------------------------------------------------------------------------|
| 1 | Browser                       | Captures microphone audio via <code>MediaRecorder</code> ; encodes WebM/Opus. |
| 2 | Browser → Server              | <code>POST /api/transcribe</code> (multipart, audio blob).                    |
| 3 | <code>browser_audio</code>    | Decodes and resamples to 16 kHz mono float32.                                 |
| 4 | <code>preprocessing</code>    | Applies RMS normalisation; records detection metadata.                        |
| 5 | <code>pipeline</code>         | Transcribes at beam 1; obtains segment log-probabilities.                     |
| 6 | <code>confidence_check</code> | Aggregates the score, decides, writes the audit row.                          |
| 7 | <code>pipeline</code>         | If rejected on first pass, re-transcribes at beam 5 and re-evaluates.         |
| 8 | Server → Browser              | Returns text, decision, score, threshold, and active device.                  |

| #  | Participant                   | Action                                                                        |
|----|-------------------------------|-------------------------------------------------------------------------------|
| 9  | Browser → Server              | If accepted: <code>POST /api/respond</code> with the message and prior turns. |
| 10 | Server → Ollama               | Proxies a streaming chat completion over loopback.                            |
| 11 | Server → Browser              | Relays tokens as a chunked text stream; the interface renders progressively.  |
| 12 | Browser → Server              | <code>POST /api/tts</code> with the completed response text.                  |
| 13 | <code>pipeline</code> → Piper | Invokes the synthesis binary as a subprocess; returns WAV bytes.              |
| 14 | Browser                       | Plays the audio and retains the blob for on-demand replay.                    |

## 7.4 Technology Selection and Rationale

Every selection below was constrained first by CON-01 (no external network calls), which excluded all hosted services outright, and second by CON-02 (4 GB of shared VRAM).

TABLE 7.4 — COMPONENT SELECTION, ALTERNATIVES CONSIDERED, AND RATIONALE

| Function           | Selected                                                             | Alternatives considered                                                                                | Rationale                                                                                                                                                                                                                                                                                                                                                               |
|--------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Speech recognition | <code>faster-whisper</code> on CTranslate2; model <code>small</code> | OpenAI <code>whisper</code> reference implementation; <code>whisper.cpp</code> ; Vosk; hosted STT APIs | Hosted services excluded by CON-01. The reference implementation requires the full PyTorch stack — several gigabytes of dependency and materially higher VRAM. CTranslate2 runs the same weights with lower memory and faster inference, and supports <code>int8_float16</code> quantisation, which was decisive under CON-02. Vosk offers lower accuracy on this task. |
| Model runtime      | Ollama 0.33.3                                                        | llama.cpp directly; LM Studio; vLLM; Hugging Face <code>transformers</code>                            | Ollama manages model acquisition, quantisation format, GPU layer offload, and memory residency automatically, exposing a stable local HTTP interface. vLLM targets server-class GPUs. Direct llama.cpp integration would have required manual CUDA build management for no functional gain.                                                                             |
| Language model     | <code>llama3.2:3b</code> (Q4_K_M, 2.0 GB)                            | <code>qwen2.5:3b-instruct</code> ; 7–8 B parameter models                                              | Selected after direct comparison (§12.6). Larger models exceed the VRAM budget when co-resident with the recogniser. Between the 3 B candidates, <code>llama3.2:3b</code> was more reliably correct on elementary arithmetic and produced phrasing better suited to spoken delivery.                                                                                    |
| Speech synthesis   | Piper; voice <code>en_US-lessac-medium</code>                        | Coqui TTS; espeak-ng; Windows SAPI                                                                     | Fast enough on CPU alone, leaving the GPU entirely to recognition and generation, while remaining natural enough for conversational use. espeak-ng is markedly more synthetic; Coqui carries a heavier dependency footprint. Licensing implications are recorded in §15.8.                                                                                              |

| Function           | Selected                           | Alternatives considered                                              | Rationale                                                                                                                                                                                                                                                                  |
|--------------------|------------------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Endpoint detection | WebRTC VAD                         | Fixed-duration recording; energy thresholding; Silero VAD            | Fixed durations either truncate long utterances or add latency to short ones. Energy thresholding fails on whispered input — a primary target condition. WebRTC VAD is well established, computationally trivial, and operates frame-by-frame at the required sample rate. |
| Signal processing  | <code>librosa</code>               | Hand-written DSP; <code>torchaudio</code> ; <code>scipy</code> alone | Provides tested implementations of pitch tracking, onset detection, and time-scale modification. <code>torchaudio</code> would reintroduce the PyTorch dependency deliberately avoided in recognition.                                                                     |
| Media decoding     | PyAV                               | Client-side WAV encoding; external <code>ffmpeg</code> binary        | Already present as a transitive dependency of the recogniser, so adds no install cost. Bundles its own FFmpeg libraries, avoiding a system-level dependency. Client-side encoding would have required hand-written resampling in JavaScript.                               |
| Web framework      | Flask; Waitress in production mode | FastAPI; Django; <code>http.server</code>                            | The HTTP surface is five endpoints; Django is disproportionate. FastAPI's asynchronous model offers no benefit where the workload is bounded by synchronous GPU inference. Waitress was selected over Gunicorn because Gunicorn does not support Windows (CON-03).         |

## 7.5 Deployment Topology

All processes execute on a single host, communicating exclusively over the loopback interface and the local filesystem.

| Process                                            | Interface                    | Notes                                                                                    |
|----------------------------------------------------|------------------------------|------------------------------------------------------------------------------------------|
| Application server<br>( <code>chat_web.py</code> ) | <code>127.0.0.1:5006</code>  | Hosts the interface and the pipeline; loads the recogniser into GPU memory on first use. |
| Ollama service                                     | <code>127.0.0.1:11434</code> | Installed and managed independently; retains the model in memory between requests.       |
| Piper                                              | Subprocess,<br>stdin/stdout  | Invoked per synthesis request; no persistent process.                                    |
| Dataset recorder<br>( <code>record_web.py</code> ) | <code>127.0.0.1:5005</code>  | Run only during dataset construction; not required at conversational runtime.            |

### DEPLOYMENT NOTE

The browser is the only component requiring a graphical session, and it connects to `localhost` . Because `getUserMedia` treats `localhost` as a secure context, microphone access functions without TLS. Any deployment reachable by hostname or public address would require HTTPS, and would additionally require the access controls that are explicitly out of scope (§2.3.2, §14.4).

# 8 Detailed Design

## 8.1 Configuration Subsystem

`src/config.py` defines a single frozen dataclass, `Config`, holding every tunable parameter in the system. A module-level singleton `CONFIG` is constructed at import time from `Config.from_env()`, which reads the process environment after `python-dotenv` has loaded any local `.env` file.

Three properties of this design matter. First, *immutability*: the dataclass is frozen, so configuration cannot drift at runtime. Second, *typed coercion with diagnostics*: helper functions `_env_int`, `_env_float` and `_env_bool` convert string environment values and raise a `ValueError` naming the offending variable if conversion fails, rather than failing obscurely later. Third, *testability*: `from_env()` is a classmethod, so tests construct isolated instances after manipulating the environment without mutating the global singleton.

The refactor that introduced this module replaced scattered `os.environ.get()` calls previously duplicated across five modules. Backwards compatibility was preserved deliberately: the environment variable names are unchanged, and `pipeline.py` retains module-level aliases (`SAMPLE_RATE`, `OLLAMA_URL`, and others) that now resolve to `CONFIG` values, so existing call sites in `cli.py` and `chat_web.py` required no modification. This confined the change to where configuration is *read*, not to how it is *used*.

## 8.2 Audio Capture and Endpoint Detection

`pipeline.capture_audio()` opens a raw 16-bit input stream at 16 kHz and consumes it in 30 ms frames (480 samples), the frame size required by the WebRTC VAD. A small state machine determines utterance boundaries:

| State       | Transition condition                 | Action                                                                                                              |
|-------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Untriggered | Frame classified as speech           | Enter Triggered; prepend the retained pre-roll buffer (10 frames, $\approx$ 300 ms) so speech onset is not clipped. |
| Untriggered | Frame classified as non-speech       | Append to a bounded ring buffer; discard the oldest frame.                                                          |
| Triggered   | Frame classified as speech           | Append frame; reset the trailing-silence counter.                                                                   |
| Triggered   | Consecutive non-speech $\geq$ 800 ms | Terminate capture; return the accumulated utterance.                                                                |
| Any         | Elapsed time > 15 s                  | Terminate capture unconditionally (FR-1.5).                                                                         |

An utterance containing fewer than 150 ms of speech frames is discarded as spurious rather than transcribed. Captured int16 samples are converted to float32 normalised to  $\pm 1.0$  for the recogniser.

## 8.3 Audio Preprocessing

### 8.3.1 Amplitude normalisation

Normalisation targets a fixed RMS level of 0.1 rather than a fixed peak. RMS was chosen deliberately: peak normalisation is driven by the single loudest sample, so a whispered clip containing one incidental transient would be scaled as though it were loud, whereas a shouted clip is loud throughout. RMS reflects sustained perceived loudness, which is the property that actually differs between whispered and shouted delivery.

Two safeguards apply. Input whose RMS falls below  $10^{-6}$  is passed through unmodified, avoiding division by near-zero and the consequent amplification of the noise floor. After gain is applied, if the resulting peak would exceed 0.99 the signal is rescaled so that it does not clip — necessary because a very quiet input attracts a large gain factor.

### 8.3.2 Pitch and rate detection

Median fundamental frequency is estimated with probabilistic YIN over voiced frames, and flagged when outside 80–400 Hz. Speaking rate is estimated from onset density — detected onsets divided by clip duration — and flagged when outside 0.75–1.35 times a nominal reference rate.

#### DESIGN STATUS — CORRECTIONS DISABLED

The corresponding *correction* stages (pitch shifting and time-scale modification) are implemented but disabled by default via `ENABLE_PITCH_CORRECTION` and `ENABLE_SPEED_CORRECTION`. Empirical evaluation demonstrated that applying them reduced recognition accuracy from 96.0% to 48.0% on real recordings. The detection and logging paths remain active because the metadata is diagnostically useful; only the audio-modifying step is gated. The full analysis is presented in §12.4, and the behaviour is protected by regression tests (§11.3).

### 8.3.3 Failure containment

The pitch and rate stages call into numba-compiled routines within `librosa`. Under memory pressure — with the recogniser, language model, and synthesiser all resident — that just-in-time compilation has been observed to raise `MemoryError`. Each stage is therefore wrapped in its own exception guard that logs the failure, records it in the preprocessing report, and returns the audio unmodified. A second guard in the orchestration loop catches anything that escapes the first. Preprocessing is an accuracy enhancement, never a precondition for a turn completing.

## 8.4 Speech Recognition and Device Selection

### 8.4.1 Verified device fallback

`load_whisper_model()` attempts GPU construction first, then executes a short trial transcription over 0.25 s of silence before returning the model. This ordering is not incidental. On Windows, `WhisperModel(device="cuda")` constructs successfully even when the CUDA runtime libraries cannot be loaded; the failure surfaces only on the first genuine inference. Validating by construction alone would therefore have produced a system that appeared healthy at start-up and failed on the user's first utterance. The trial inference converts that latent fault into a deterministic start-up check, after which the loader falls back to CPU with an `int8` compute type. The same pattern is applied in `verify_setup.py` and `benchmark.py`.

## 8.4.2 CUDA library resolution on Windows

`src/cuda_dlls.py` exists to solve a specific, non-obvious platform defect. The CTranslate2 backend requires cuBLAS and cuDNN. On Linux the corresponding `nvidia-*-cu12` wheels are sufficient on their own, because the shared library carries an `RPATH` referring to them. Windows has no `RPATH` equivalent, and CTranslate2 resolves these libraries through a plain `LoadLibrary` call — which does *not* consult directories registered via `os.add_dll_directory()`, the conventional remedy for this class of problem. That approach is silently ineffective here.

The module therefore locates each installed `nvidia.*` package, derives its `bin` directory, and prepends it to the process `PATH` before `faster_whisper` is imported anywhere. Every entry point that loads the recogniser calls this function ahead of the relevant import. Without it, the observed failure is `RuntimeError: Library cublas64_12.dll is not found or cannot be loaded` at first inference.

## 8.4.3 Two-pass decoding

`transcribe()` performs a single decode at a caller-supplied beam size, returning both the joined text and the raw segment objects, which carry the log-probabilities the confidence layer requires. `transcribe_with_confidence()` composes this with the gating policy: a fast first pass at beam 1, and — only where the first pass is judged insufficient — a second pass at beam 5. The wider search is thus paid for only when it is needed, rather than on every utterance.

## 8.5 Confidence Decision Layer

### 8.5.1 Score aggregation

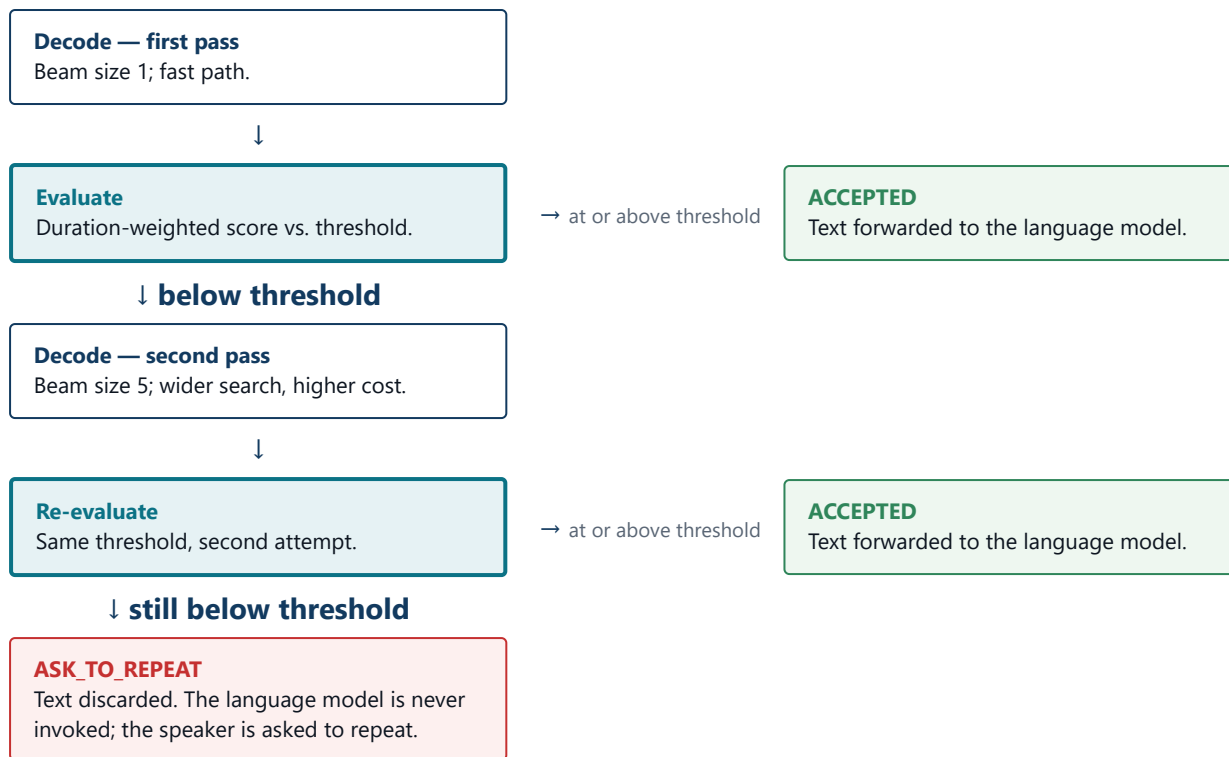
The recogniser reports `avg_logprob` per segment. These are combined into a single utterance score as a duration-weighted mean, so that a long confident segment is not outweighed by a short uncertain one. An utterance producing no segments returns `-999.0`, a sentinel far below any plausible threshold, ensuring that silence or undecodable audio is always rejected rather than accidentally accepted.

### 8.5.2 Decision function

TABLE 8.1 — CONFIDENCE GATE DECISION MATRIX

| Score vs. threshold   | Attempt         | Decision         | Consequence                                                                |
|-----------------------|-----------------|------------------|----------------------------------------------------------------------------|
| At or above threshold | Either          | ACCEPTED         | Text forwarded to the language model.                                      |
| Below threshold       | First (beam 1)  | RETRY_MORE_BEAMS | Re-decode at beam 5; evaluate again.                                       |
| Below threshold       | Second (beam 5) | ASK_TO_REPEAT    | Text discarded; user asked to repeat. The language model is never invoked. |

The comparison is inclusive (`score >= threshold`), so a score exactly at the threshold is accepted; this boundary is covered by an explicit unit test.



**Figure 2.** Confidence-gating decision state machine. Every transition — including both acceptance paths — appends a row to the audit trail described in §8.5.4. The wider second-pass search is executed only for utterances that fail the first evaluation, so its cost is not incurred on the common path.

### 8.5.3 Threshold selection and calibration

The default threshold is `-0.6`. `avg_logprob` is not a calibrated probability, so the value was chosen from observed behaviour on this project’s own recordings, where three bands were consistently apparent: above `-0.5` the recogniser was essentially always correct; between `-0.5` and `-0.9` it was unreliable; below `-0.9` it was typically wrong. Representative values observed during system testing were `-0.29` for a clean utterance, `-0.60` for a borderline case that was accepted on the second pass, and `-0.85` and `-0.88` for inputs correctly rejected.

#### CALIBRATION GUIDANCE

This distribution shifts with speaker, microphone, and acoustic environment. The `confidence_log.csv` audit trail exists precisely so the threshold can be re-tuned from evidence: too many spurious repeat requests indicates the threshold should be lowered (more negative); incorrect transcriptions reaching the model indicates it should be raised. This is a known limitation and a documented fairness consideration (§15.3).

### 8.5.4 Audit trail

Every decision — including every acceptance — appends one row to `confidence_log.csv`, recording UTC timestamp, transcribed text, score to four decimal places, threshold in force, decision, beam size used, and attempt number. Logging acceptances as well as rejections is deliberate: a log of failures alone cannot establish a false-rejection rate.

## 8.6 Language Model Integration

### 8.6.1 Response shaping

A system prompt constrains responses to one to three plain conversational sentences and prohibits lists, headings, and markup. This is not stylistic preference but a functional requirement of the medium: every response is read aloud, where a numbered list becomes an unstructured monologue with none of the visual scaffolding that made the structure useful. During model comparison it emerged that this prompt constraint improved spoken suitability far more than the choice of model did — both candidate models produced structured, list-heavy output without it, and concise spoken prose with it (§12.6).

### 8.6.2 Transport and context

Two integration paths exist. `pipeline.generate_response()` targets the single-turn completion endpoint and is used by the terminal interface. The browser interface instead proxies the chat endpoint through `chat_web.respond_endpoint()`, supplying the system prompt followed by the trailing window of conversation history (bounded at twelve messages, approximately six exchanges) and the new user message. This gives the browser interface genuine multi-turn memory, which the terminal interface does not have — a deliberate asymmetry, since the browser client can hold session state that a stateless terminal loop cannot.

Both paths stream. Tokens are relayed to the caller as they arrive rather than buffered until completion, so perceived latency is governed by time-to-first-token.

### 8.6.3 Retry semantics

Connection failures are retried up to twice with linear backoff, but only under a specific condition: no output may yet have been delivered to the caller. Once a partial response has reached the user, restarting the request would duplicate content that has already been displayed or spoken, so the failure is surfaced instead. This condition is tracked explicitly in both implementations.

## 8.7 Speech Synthesis

`synthesize_speech()` invokes the Piper executable as a subprocess, writing the text to standard input and receiving a WAV file at a specified path. The executable is located relative to `sys.executable` rather than relying on `PATH`, because the console script is installed alongside the interpreter in the virtual environment and is only on `PATH` if that environment has been activated in the calling shell. Resolving relative to the running interpreter makes invocation work identically whether or not the environment was activated — a concrete robustness improvement that removed a real class of setup failure. A non-zero exit status raises a `RuntimeError` carrying the captured standard error.

## 8.8 Web Layer

### 8.8.1 Shared concerns

`web_common.py` supplies both Flask applications with identical error semantics and serving behaviour. Handlers for 400, 404, 413 and 500 return a uniform `{"error": "..."}` JSON body in place of Flask's default HTML error pages, so a programmatic client receives a parseable response in every case, and internal paths or stack

traces are never disclosed. `run_app()` selects between the Flask development server and Waitress according to `CONFIG.production`, allowing a production-grade WSGI server to be used without any code change.

### 8.8.2 Conversational service

`chat_web.py` exposes four endpoints (§10.1). The recogniser is loaded lazily on first transcription rather than at import, so start-up is immediate and a model-loading failure presents as a clear HTTP error rather than a server that never becomes ready. Request bodies are capped at 25 MB. The transcription endpoint mirrors the orchestration core exactly — it calls the same `normalize_audio()` and `transcribe_with_confidence()` functions the terminal interface uses, including the identical preprocessing fallback guard — so the two interfaces cannot diverge in behaviour.

### 8.8.3 Dataset recorder

`record_web.py` serves the recording interface and maintains the dataset manifest. The plan endpoint returns the full grid of word and modulation pairs annotated with completion state derived from the manifest, enabling the interface to resume at the first incomplete pair after an interruption. Uploads are decoded, validated for minimum duration, written as 16 kHz mono WAV, and the manifest row is replaced idempotently so that a re-recording overwrites rather than duplicates.

## 8.9 Error Handling Strategy

The governing principle is that enhancement failures degrade silently while dependency failures are surfaced explicitly and actionably.

TABLE 8.2 — FAILURE MODES AND DESIGNED RESPONSES

| Failure                                | Classification     | Designed response                                                                  |
|----------------------------------------|--------------------|------------------------------------------------------------------------------------|
| GPU unusable or CUDA libraries absent  | Degradable         | Trial inference fails; loader falls back to CPU automatically and logs the reason. |
| Preprocessing stage raises             | Degradable         | Logged; unmodified audio proceeds. Two independent guards.                         |
| Recognition confidence below threshold | Expected condition | Retry at wider beam, then request repetition. Never a silent guess.                |
| Language model server unreachable      | Dependency         | Bounded retry, then a diagnostic naming the endpoint and the remedy.               |
| Synthesis subprocess fails             | Dependency         | HTTP 500 with the captured error; the textual response remains available.          |
| Microphone absent or permission denied | Dependency         | Preflight check reports it before the session begins, with corrective guidance.    |
| Voice model file missing               | Dependency         | Preflight check names the expected path and the download source.                   |
| Upload exceeds size cap                | Client error       | HTTP 413 with a JSON body stating the limit.                                       |
| Invalid configuration value            | Fatal, early       | <code>ValueError</code> at start-up naming the offending variable.                 |

## 9 Data Design

### 9.1 Data Inventory

| Artefact                                          | Format              | Lifetime               | Contents and sensitivity                                                                        |
|---------------------------------------------------|---------------------|------------------------|-------------------------------------------------------------------------------------------------|
| <code>data/test_samples/*.wav</code>              | WAV, 16 kHz mono    | Persistent, local only | Recorded speech. <b>Personal data.</b> Excluded from version control.                           |
| <code>data/test_samples/manifest.csv</code>       | CSV                 | Persistent             | Filenames and ground-truth labels. No audio content; safe to commit.                            |
| <code>data/results/benchmark_results.csv</code>   | CSV                 | Persistent             | Per-clip transcriptions and correctness under both conditions. Contains transcribed words only. |
| <code>data/results/accuracy_comparison.png</code> | PNG                 | Persistent             | Rendered aggregate results. No personal data.                                                   |
| <code>logs/confidence_log.csv</code>              | CSV                 | Persistent, unrotated  | Every gating decision, <b>including transcribed text.</b> Excluded from version control.        |
| <code>logs/pipeline.log</code>                    | Text                | Rotating, 5 MB × 3     | Stage timings and diagnostics. Excluded from version control.                                   |
| Conversation history                              | In-memory (browser) | Session only           | Discarded on page reload. Never written to disk.                                                |
| Synthesis output                                  | WAV, temporary      | Overwritten per turn   | Transient artefact of the current response.                                                     |

### 9.2 Schemas

TABLE 9.1 — **MANIFEST.CSV**

| Field                      | Type   | Description                                                                                                          |
|----------------------------|--------|----------------------------------------------------------------------------------------------------------------------|
| <code>filename</code>      | string | Clip filename, formatted <code>{word}_{modulation}.wav</code> .                                                      |
| <code>expected_word</code> | string | Ground-truth label used to score recognition.                                                                        |
| <code>modulation</code>    | enum   | One of <code>normal</code> , <code>whispered</code> , <code>shouted</code> , <code>slow</code> , <code>fast</code> . |

TABLE 9.2 — **CONFIDENCE\_LOG.CSV**

| Field                  | Type          | Description                  |
|------------------------|---------------|------------------------------|
| <code>timestamp</code> | ISO 8601, UTC | Time the decision was taken. |

| Field                       | Type         | Description                                                                             |
|-----------------------------|--------------|-----------------------------------------------------------------------------------------|
| <code>text</code>           | string       | Transcription under evaluation.                                                         |
| <code>avg_logprob</code>    | float (4 dp) | Duration-weighted mean segment log-probability.                                         |
| <code>threshold</code>      | float        | Threshold in force at the time of the decision.                                         |
| <code>decision</code>       | enum         | <code>accepted</code> , <code>retry_more_beams</code> , or <code>ask_to_repeat</code> . |
| <code>beam_size_used</code> | integer      | Decoder beam width for this attempt.                                                    |
| <code>attempt</code>        | integer      | Attempt index within the utterance (1 or 2).                                            |

TABLE 9.3 — `BENCHMARK_RESULTS.CSV`

| Field                                                                           | Type    | Description                                                            |
|---------------------------------------------------------------------------------|---------|------------------------------------------------------------------------|
| <code>filename</code> , <code>expected_word</code> ,<br><code>modulation</code> | string  | Carried through from the manifest.                                     |
| <code>raw_transcription</code>                                                  | string  | Transcription without preprocessing.                                   |
| <code>raw_correct</code>                                                        | boolean | Whether the expected word appears in the normalised raw transcription. |
| <code>preprocessed_transcription</code>                                         | string  | Transcription with preprocessing applied.                              |
| <code>preprocessed_correct</code>                                               | boolean | Correctness under the preprocessed condition.                          |

### 9.3 Design Rationale — No Database

The system uses flat files exclusively. This is a considered decision rather than an omission. The data volumes are trivial (tens of rows), the access patterns are append-only or whole-file, and every persistent artefact benefits from being directly inspectable — the audit log and benchmark results are evidence intended to be read by a human, opened in a spreadsheet, or version-controlled. A database would add an operational dependency and a schema migration burden for no functional gain at this scale. The consequence is accepted and recorded: there is no durable conversation history (§17).

### 9.4 Retention

Nothing is deleted automatically except through log rotation, which retains approximately 20 MB of pipeline logs across four generations. The confidence log grows without bound and contains transcribed speech; §15.2 addresses the resulting retention obligation, and §17 records it as a known limitation.

## 10 Interface Specification

### 10.1 Conversational Service HTTP API

Base address `http://127.0.0.1:5006`. All error responses share the body shape `{"error": "<description>"}`.

TABLE 10.1 — `CHAT_WEB.PY` ENDPOINTS

| Method | Path                                        | Request                                                                                                                      | Response                                                                                                                                                                                                                                                         |
|--------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/api/health</code>                    | —                                                                                                                            | <code>200</code> · <code>{ok, whisper_device}</code> . <code>whisper_device</code> is <code>null</code> until the recogniser has been loaded lazily.                                                                                                             |
| POST   | <code>/api/transcribe</code>                | <code>multipart/form-data</code> ; field <code>audio</code> containing a WebM/Opus or Ogg/Opus blob.                         | <code>200</code> · <code>{text, decision, avg_logprob, threshold, accepted, whisper_device}</code> .<br><code>400</code> if the field is absent, the media cannot be decoded, or the clip is shorter than 200 ms.<br><code>413</code> if the body exceeds 25 MB. |
| POST   | <code>/api/respond</code>                   | <code>application/json</code> · <code>{message, history[]}</code> where each history entry is <code>{role, content}</code> . | <code>200</code> · <code>text/plain</code> chunked stream of response tokens. Unreachable-model conditions are emitted as a bracketed diagnostic within the stream, since headers are already committed.<br><code>400</code> if <code>message</code> is empty.   |
| POST   | <code>/api/tts</code>                       | <code>application/json</code> · <code>{text}</code> .                                                                        | <code>200</code> · <code>audio/wav</code> byte stream.<br><code>400</code> if <code>text</code> is empty.<br><code>500</code> if synthesis fails.                                                                                                                |
| GET    | <code>/</code> , <code>/&lt;file&gt;</code> | —                                                                                                                            | Static interface assets.                                                                                                                                                                                                                                         |

### 10.2 Dataset Recorder HTTP API

TABLE 10.2 — `RECORD_WEB.PY` ENDPOINTS

| Method | Path                     | Request                                                                                               | Response                                                                                                                                                                                |
|--------|--------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET    | <code>/api/plan</code>   | Optional <code>?words=</code> comma-separated override.                                               | <code>200</code> · <code>{combos[], words[], modulations[]}</code> ; each combo carries a <code>done</code> flag enabling resume.                                                       |
| POST   | <code>/api/upload</code> | <code>multipart/form-data</code> · <code>word</code> , <code>modulation</code> , <code>audio</code> . | <code>200</code> · <code>{ok, filename, duration_s}</code> .<br><code>400</code> on invalid label, absent audio, decode failure, or clip under 300 ms.<br><code>413</code> above 25 MB. |
| GET    | <code>/api/health</code> | —                                                                                                     | <code>200</code> · <code>{ok}</code> .                                                                                                                                                  |

## 10.3 External Interfaces

| Interface           | Mechanism                                              | Contract                                                                                                                                                                                                                       |
|---------------------|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ollama — completion | <code>POST /api/generate</code> over loopback          | Request carries <code>model</code> , <code>prompt</code> , <code>system</code> , <code>stream</code> . Response is newline-delimited JSON; each object supplies a <code>response</code> fragment and a <code>done</code> flag. |
| Ollama — chat       | <code>POST /api/chat</code> over loopback              | Request carries <code>model</code> and a <code>messages</code> array of role/content pairs. Response fragments arrive under <code>message.content</code> .                                                                     |
| Ollama — inventory  | <code>GET /api/tags</code>                             | Used by preflight checks to confirm the configured model is present.                                                                                                                                                           |
| Piper               | Subprocess; text on stdin                              | Arguments <code>--model</code> and <code>--output_file</code> . Exit status zero indicates success; stderr is captured for diagnostics.                                                                                        |
| Browser capture     | <code>getUserMedia</code> + <code>MediaRecorder</code> | Negotiates the first supported MIME type from a preference list, covering both Chromium (WebM/Opus) and Gecko (Ogg/Opus).                                                                                                      |

## 10.4 Configuration Interface

TABLE 10.3 — ENVIRONMENT VARIABLES (ALL OPTIONAL; DEFAULTS SHOWN)

| Variable                          | Default                                            | Effect                                                                     |
|-----------------------------------|----------------------------------------------------|----------------------------------------------------------------------------|
| <code>WHISPER_MODEL_SIZE</code>   | <code>small</code>                                 | Recognition model size.                                                    |
| <code>CONFIDENCE_THRESHOLD</code> | <code>-0.6</code>                                  | Gate acceptance threshold.                                                 |
| <code>CONFIDENCE_LOG_PATH</code>  | <code>logs/confidence_log.csv</code>               | Audit trail location.                                                      |
| <code>OLLAMA_URL</code>           | <code>http://localhost:11434</code>                | Model server address.                                                      |
| <code>OLLAMA_MODEL</code>         | <code>llama3.2:3b</code>                           | Model identifier.                                                          |
| <code>PIPER_EXECUTABLE</code>     | Resolved from <code>sys.executable</code>          | Synthesis binary path.                                                     |
| <code>PIPER_MODEL_PATH</code>     | <code>models/piper/en_US-lessac-medium.onnx</code> | Voice model path.                                                          |
| <code>LOG_DIR</code>              | <code>logs</code>                                  | Log directory.                                                             |
| <code>CHAT_WEB_PORT</code>        | <code>5006</code>                                  | Conversational service port.                                               |
| <code>RECORD_WEB_PORT</code>      | <code>5005</code>                                  | Recorder service port.                                                     |
| <code>PRODUCTION</code>           | <code>0</code>                                     | When truthy, serves via Waitress rather than the Flask development server. |

## 11 Verification & Validation

### 11.1 Strategy

Verification operates at three levels. *Automated unit and integration tests* cover deterministic logic — confidence arithmetic and decisions, configuration parsing, HTTP endpoint contracts, and signal-processing invariants. *Manual system testing* covers behaviour that requires real hardware: microphone capture, GPU inference, model serving, and audio playback. *Empirical measurement* (§12) covers the accuracy claims, which cannot be established by testing alone because they are statements about a model's behaviour on real data.

A deliberate constraint governs the automated suite: it must run with no GPU, no model server, no microphone, and no synthesis binary. Every external interaction is mocked at its call site. This keeps the suite fast, deterministic, and executable on ordinary continuous-integration infrastructure — at the cost, honestly acknowledged, that it verifies the system's *logic* rather than its integration with those external components. Integration is covered by §11.5.

### 11.2 Automated Test Inventory

TABLE 11.1 — AUTOMATED TEST SUITE COMPOSITION (42 TESTS, ALL PASSING AT BASELINE)

| Module                                      | Tests | Coverage                                                                                                                                                                                                                                         |
|---------------------------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tests/test_confidence_check.py</code> | 10    | Score aggregation including empty and zero-duration segments; duration weighting; all three decision branches; threshold boundary inclusivity; audit row content; append rather than overwrite semantics.                                        |
| <code>tests/test_preprocessing.py</code>    | 9     | RMS normalisation reaches target; near-silence does not amplify; large gain does not clip; <b>correction-disabled regression tests</b> (§11.3); end-to-end robustness on short, silent, and tonal input.                                         |
| <code>tests/test_config.py</code>           | 10    | Defaults; immutability enforcement; string, float, integer and boolean overrides; invalid-value diagnostics; equivalence of a clean environment to bare defaults.                                                                                |
| <code>tests/test_chat_web.py</code>         | 13    | Health contract; transcription success, rejection surfacing, decode failure, absent file, and under-length rejection; synthesis success and failure; streaming relay; history forwarding order; unreachable-server handling without retry delay. |

## 11.3 Regression Tests of Record

### EVIDENCE LOCK-IN

Two tests in `tests/test_preprocessing.py` exist specifically to prevent the defect described in §12.4 from recurring. Each forces the detection stage to report a clear outlier — an implausible fundamental frequency, or a single onset across a long clip — and then asserts that the corresponding correction function ( `pitch_shift` , `time_stretch` ) is **never invoked** and that the returned audio is bit-identical to the input. They also assert that both feature flags remain `False` .

The intent is procedural as much as technical: re-enabling either correction now requires deliberately editing a failing test, which forces the change to be justified against new measurement rather than made casually.

## 11.4 Continuous Integration

The workflow at `.github/workflows/tests.yml` executes on every push and pull request. It provisions Python 3.11 on `ubuntu-latest` , installs `requirements-dev.txt` — deliberately *not* the GPU requirements, since roughly 1.3 GB of CUDA wheels serve no purpose where every GPU path is mocked — then runs `ruff check` . followed by `pytest -v` . Lint findings and test failures both fail the build.

## 11.5 System Testing Performed

TABLE 11.2 — MANUAL VERIFICATION RECORD

| ID    | Test                                                                                  | Result                                                                                                              |
|-------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| ST-01 | Installation verification across all four dependencies.                               | Pass — 4/4 reported healthy, re-executed after the configuration refactor to confirm no regression.                 |
| ST-02 | End-to-end pipeline on synthesised input: recognition, gating, generation, synthesis. | Pass — all stages completed; recognition executed on CUDA in 0.85 s.                                                |
| ST-03 | Confidence rejection path with non-speech input.                                      | Pass — spurious transcription scored -0.88, correctly refused, repeat request issued; no model invocation occurred. |
| ST-04 | Co-residency of recogniser and language model within the VRAM budget.                 | Pass — 2 592 MiB peak of 4 096 MiB available.                                                                       |
| ST-05 | Multi-turn context retention through the chat endpoint.                               | Pass — a follow-up question referencing the prior turn was answered correctly from history.                         |
| ST-06 | Production serving mode via Waitress.                                                 | Pass — confirmed by the <code>Server: waitress</code> response header; endpoints functioned unchanged.              |
| ST-07 | Upload size enforcement using a 26 MB payload against the 25 MB cap.                  | Pass — HTTP 413 with the expected JSON body.                                                                        |

| ID    | Test                                                                | Result                                                                                                                               |
|-------|---------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| ST-08 | Uniform JSON error shape on an unknown route.                       | Pass — HTTP 404 returned <code>{"error": "Not found."}</code> rather than an HTML page.                                              |
| ST-09 | Browser interface state transitions under a synthetic media stream. | Pass — two CSS defects and one event-ordering defect were identified and corrected during this test.                                 |
| ST-10 | Recorder resume after interruption.                                 | Pass — completed clips detected; session resumed at the first incomplete pair.                                                       |
| ST-11 | Live conversational session by the primary user.                    | Pass with observation — one presentation defect (device badge displaying a placeholder before lazy load) was reported and corrected. |

## 11.6 Requirements Traceability

TABLE 11.3 — TRACEABILITY FROM REQUIREMENT TO EVIDENCE

| Requirement group                       | Verification method              | Evidence                                                          |
|-----------------------------------------|----------------------------------|-------------------------------------------------------------------|
| FR-1.x Capture                          | Demonstration, inspection        | ST-02, ST-11; capture state machine reviewed (§8.2).              |
| FR-2.x Preprocessing                    | Unit and regression tests        | <code>test_preprocessing.py</code> (9 tests), §11.3.              |
| FR-3.x Recognition                      | Demonstration, unit tests        | ST-01, ST-02, ST-04; device fallback verified in situ (§8.4.1).   |
| FR-4.x Confidence gating                | Unit tests, demonstration        | <code>test_confidence_check.py</code> (10 tests), ST-03.          |
| FR-5.x Generation                       | Integration tests, demonstration | <code>test_chat_web.py</code> streaming and history tests, ST-05. |
| FR-6.x, FR-7.x Synthesis and interfaces | Demonstration                    | ST-02, ST-09, ST-11.                                              |
| FR-8.x Evaluation tooling               | Demonstration, inspection        | ST-10; benchmark artefacts REF-02 and REF-03.                     |
| FR-9.x Configuration and operations     | Unit tests, demonstration        | <code>test_config.py</code> (10 tests), ST-01, ST-06.             |
| NFR-P.x Performance                     | Measurement                      | §13; ST-02, ST-04.                                                |
| NFR-R.x Reliability                     | Unit tests, inspection           | Fallback guards (§8.9); preprocessing containment tests.          |
| NFR-S.x Security                        | Demonstration, inspection        | ST-07, ST-08; bind address and exclusion rules inspected.         |
| NFR-M.x Maintainability                 | Automated                        | CI pipeline: 42 tests plus lint, green at baseline.               |

## 12 Experimental Evaluation

This section reports the empirical evaluation of the system’s central design hypothesis. The result was negative: the preprocessing layer, as originally implemented, substantially degraded recognition accuracy rather than improving it. The hypothesis, the measurement, the root-cause analysis, the corrective action, and the re-validation are reported in full.

### 12.1 Hypothesis

The design premise stated in §3.1 was that normalising loudness, pitch, and speaking rate before recognition would reduce the acoustic distance between modulated and typical speech, and would therefore improve recognition accuracy under whispered, shouted, slow, and fast delivery. The benchmark was constructed specifically so that this claim could be falsified rather than assumed.

### 12.2 Method

| Aspect               | Definition                                                                                                                                                               |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Design               | Within-subject, two-condition comparison. Every clip is transcribed twice — once unprocessed, once preprocessed — with all other parameters held constant.               |
| Independent variable | Application of the preprocessing layer (absent / present).                                                                                                               |
| Dependent variable   | Word recognition accuracy: the proportion of clips whose normalised transcription contains the expected word.                                                            |
| Controlled           | Identical model ( <code>small</code> ), identical compute type, identical beam size (5 in both conditions), identical language setting, identical audio source.          |
| Scoring              | Case-insensitive; punctuation stripped; the expected word must appear as a discrete token. This tolerates trailing punctuation and filler while rejecting substitutions. |
| Execution            | <code>python src/benchmark.py</code> . Results written to <code>benchmark_results.csv</code> ; chart rendered to <code>accuracy_comparison.png</code> .                  |

### 12.3 Dataset

A purpose-recorded corpus of 25 clips: five command words — *yes*, *no*, *stop*, *help*, *start* — each spoken in five modulations: normal, whispered, shouted, slow, and fast. All clips were recorded by a single speaker through the browser recorder in a quiet domestic environment, at 16 kHz mono. The corpus is small, and §12.7 addresses the resulting limits on inference.

### 12.4 Result and Root-Cause Analysis

The first execution against real recordings produced the following:

TABLE 12.1 — INITIAL BENCHMARK RESULT (PREPROCESSING AS ORIGINALLY IMPLEMENTED)

| Modulation | Raw accuracy | With preprocessing | Delta |
|------------|--------------|--------------------|-------|
| Normal     | 100.0%       | 40.0%              | −60.0 |
| Whispered  | 100.0%       | 60.0%              | −40.0 |
| Shouted    | 100.0%       | 40.0%              | −60.0 |
| Slow       | 80.0%        | 60.0%              | −20.0 |
| Fast       | 100.0%       | 40.0%              | −60.0 |
| Overall    | 96.0%        | 48.0%              | −48.0 |

#### NEGATIVE RESULT

Preprocessing halved recognition accuracy. The degradation was not confined to the modulated conditions it was designed to assist — accuracy under *normal* delivery fell by 60 points, which is definitive evidence of a defect rather than a tuning deficiency. Representative failures included *stop* transcribed as “So...”, *help* as “L L L”, and one clip returning an empty transcription.

### 12.4.1 Ablation

Rather than adjusting parameters speculatively, the contribution of each stage was isolated by disabling stages individually and re-running the full corpus:

TABLE 12.2 — CONTROLLED ABLATION OF PREPROCESSING STAGES

| Configuration                                 | Accuracy | Interpretation                                                                                               |
|-----------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------|
| No preprocessing (baseline)                   | 96.0%    | Reference condition.                                                                                         |
| Volume + pitch + speed (as shipped initially) | 48.0%    | Severe degradation.                                                                                          |
| Volume + pitch (speed correction disabled)    | 92.0%    | Recovers most of the loss: speed correction is the dominant cause.                                           |
| Volume only (pitch and speed disabled)        | 96.0%    | Exact parity with baseline: pitch correction is a secondary contributor; amplitude normalisation is neutral. |

### 12.4.2 Cause — speed correction

Speaking rate was estimated as onset density: detected onsets divided by total clip duration, compared against a nominal reference. This measure is only meaningful over continuous multi-syllable speech, where enough onsets accumulate to form a stable estimate. Applied to an isolated single command word it fails in two compounding ways. The word contributes approximately one onset, so the numerator is a sample of size one. The denominator is the entire recorded clip, most of which is silence surrounding the word — so the estimate is governed principally by how much padding the recording happens to contain, not by how quickly the speaker spoke.

The consequence was systematic misclassification. Every clip, in every modulation, was labelled “slow”. The clearest illustration is `start_fast`, genuinely spoken quickly, which was assigned a rate ratio of 0.22 — classified as extremely slow — and then time-stretched *further in the wrong direction*, producing audio degraded past intelligibility and an empty transcription. The design defect is precisely the violation of assumption ASM-03: a technique valid for continuous speech was applied to isolated words.

12.4.3 Cause — pitch correction

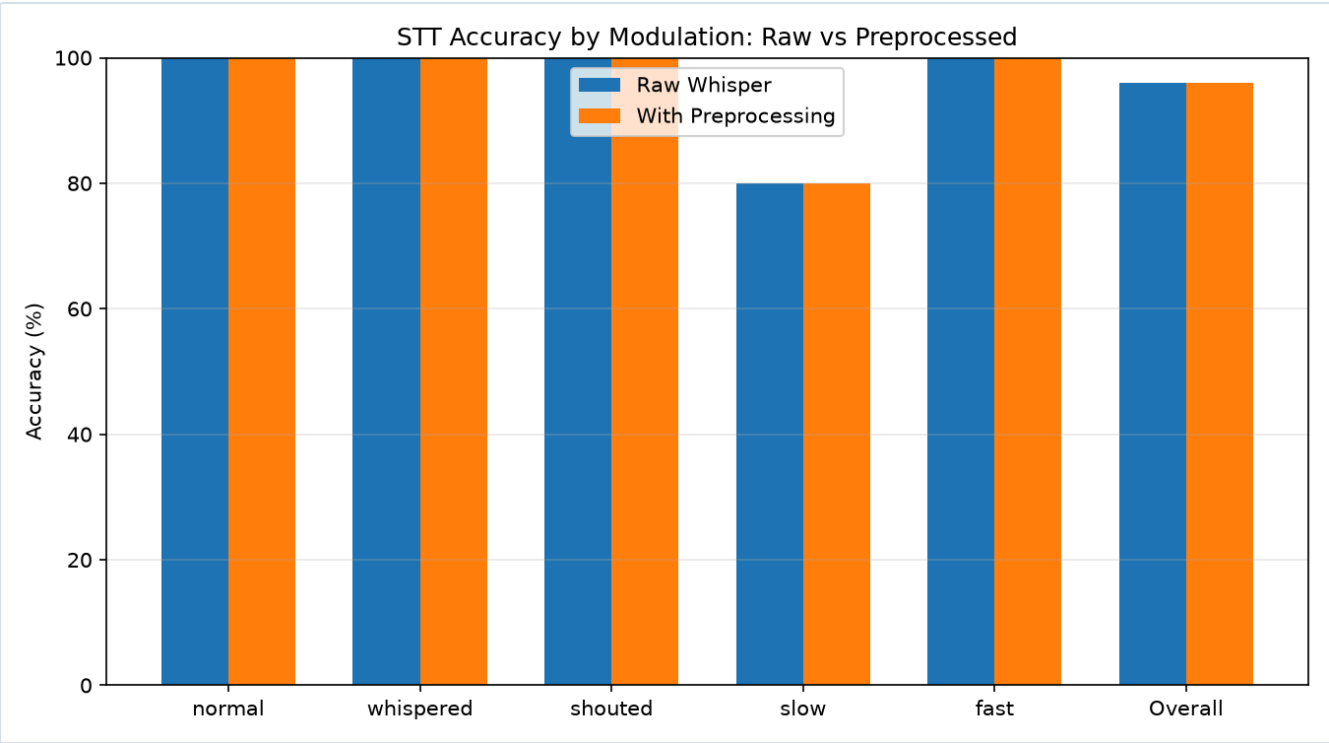
Pitch tracking on short, consonant-heavy words proved unreliable. On one *start* clip the tracker reported a median fundamental of 766 Hz — implausible for this speaker and consistent with locking onto a plosive or fricative burst rather than voiced phonation. The system then “corrected” a pitch anomaly that did not exist, damaging a word that required no intervention.

12.5 Corrective Action and Re-validation

Both correction stages were disabled by default through explicit feature flags. Detection and logging were retained, since the metadata remains diagnostically useful and carries no risk when it does not modify the signal. The benchmark was then re-executed unchanged:

TABLE 12.3 — BENCHMARK RESULT AFTER CORRECTIVE ACTION (CURRENT BASELINE)

| Modulation | Raw accuracy | With preprocessing | Delta |
|------------|--------------|--------------------|-------|
| Normal     | 100.0%       | 100.0%             | 0.0   |
| Whispered  | 100.0%       | 100.0%             | 0.0   |
| Shouted    | 100.0%       | 100.0%             | 0.0   |
| Slow       | 80.0%        | 80.0%              | 0.0   |
| Fast       | 100.0%       | 100.0%             | 0.0   |
| Overall    | 96.0%        | 96.0%              | 0.0   |



**Figure 3.** Recognition accuracy by modulation, raw versus preprocessed, after corrective action. Parity across all conditions confirms that the remaining preprocessing stage (amplitude normalisation) introduces no regression. Generated by `src/benchmark.py`; source data at REF-02.

The single residual failure is `help_slow`, transcribed as “L” under both conditions. Because it fails identically with and without preprocessing, it is a limitation of the recognition model on that particular utterance, not an artefact of the pipeline. It is retained in the corpus rather than removed.

### 12.6 Secondary Evaluation — Language Model Selection

Two candidate models of comparable size were compared directly under identical prompts.

| Probe                                     | 11ama3.2:3b                                         | qwen2.5:3b-instruct                                                                                                        |
|-------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Arithmetic (17 × 24)                      | 408 — correct                                       | 388 — <b>incorrect</b> on first trial; correct on a later trial, indicating instability rather than consistent capability. |
| Explanatory prose, unconstrained          | Correct but verbose                                 | Correct and more concise                                                                                                   |
| Conversational suitability, unconstrained | Four-point numbered list — unusable when read aloud | Terse to the point of curtness                                                                                             |
| Same probe, with system prompt applied    | Natural spoken phrasing, numbers verbalised         | Correct but clipped                                                                                                        |

## FINDING

The dominant factor in spoken-response quality was not model selection but prompt construction. Both models produced output unsuitable for audio delivery until the system prompt constrained form; both produced acceptable output afterwards. `llama3.2:3b` was selected on arithmetic reliability and more natural spoken phrasing, but the generalisable conclusion is that medium-appropriate prompt design outweighed a model swap at this scale.

## 12.7 Threats to Validity

| Threat                      | Type                   | Assessment                                                                                                                                                                                                                                                                                                    |
|-----------------------------|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Small corpus (n = 25)       | Statistical conclusion | A single clip shifts overall accuracy by 4 percentage points and a per-modulation figure by 20. The 48-point degradation is far outside this margin and is therefore robust; the post-correction parity is correspondingly less precise, establishing “no material regression” rather than exact equivalence. |
| Single speaker              | External               | Results characterise this system with this voice. Accuracy and the appropriate confidence threshold may differ for other speakers, accents, and vocal ranges. This is also a fairness concern (§15.3).                                                                                                        |
| Isolated words only         | External               | Findings do not transfer to continuous speech. Notably, the failure mode identified in §12.4.2 would not necessarily occur with multi-syllable utterances — the ablation justifies disabling the correction <i>for this workload</i> , not condemning the technique generally.                                |
| Quiet recording environment | External               | Robustness under background noise, reverberation, or competing speakers is untested.                                                                                                                                                                                                                          |
| Substring-based scoring     | Construct              | Scoring rewards presence of the expected token, not exact transcription. Appropriate for command-word recognition; it would be inadequate for measuring full-sentence accuracy, where word error rate would be required.                                                                                      |
| Non-blinded execution       | Internal               | Conditions were executed by the same automated script within one run, with the scoring function identical across conditions, which mitigates differential handling. No manual judgement entered the scoring.                                                                                                  |

## 12.8 Conclusions Drawn

1. The recognition model is already substantially robust to vocal modulation on isolated command words: 96% accuracy with no preprocessing whatsoever, and perfect accuracy in four of five conditions.
2. Amplitude normalisation is safe but, on this corpus, unnecessary — the model handled the loudness range unaided. It is retained because it is neutral and would plausibly matter under a wider dynamic range than this corpus exhibits.
3. Pitch and rate correction, as implemented, were actively harmful for this workload and have been disabled with the evidence recorded and a regression test enforcing the decision.
4. The residual robustness value of the system therefore lies in the confidence-gating layer, which addresses the failure mode identified in §3.1 directly and is not captured by this accuracy benchmark.

5. Most significantly for engineering practice: the benchmark earned its cost by falsifying the project's own design hypothesis. Had it not been built, a plausible but harmful component would have shipped with an unverified claim of improvement attached to it.

# 13 Performance Characteristics

## 13.1 Latency

Measured on the reference configuration (§3.4). Figures are indicative single-run observations from system testing, not statistically-sampled distributions; they are reported as such.

| Stage                              | Observed           | Notes                                                                          |
|------------------------------------|--------------------|--------------------------------------------------------------------------------|
| Audio capture                      | Speaker-determined | Terminated ~800 ms after speech ends; not a processing cost.                   |
| Preprocessing                      | < 0.1 s            | Amplitude normalisation only; detection stages dominate but remain sub-second. |
| Recognition (GPU, beam 1)          | 0.85 s             | Short utterance, <code>small</code> model, <code>int8_float16</code> .         |
| Recognition (CPU fallback)         | Several seconds    | Functional but materially slower; fallback path only.                          |
| Generation — first response (cold) | 11–20 s            | Includes one-time model residency load.                                        |
| Generation — subsequent (warm)     | 1–3 s              | Perceived latency lower still, as output streams.                              |
| Synthesis                          | ~1 s               | CPU-bound; scales with response length.                                        |
| Recogniser load (first request)    | One-time           | Deferred to first use; signalled to the user rather than blocking start-up.    |

## 13.2 Resource Utilisation

TABLE 13.1 — VRAM BUDGET UNDER THE 4 096 MIB CONSTRAINT (CON-02)

| Condition                                    | VRAM in use | Notes                                                          |
|----------------------------------------------|-------------|----------------------------------------------------------------|
| Language model resident, recogniser unloaded | 2 154 MiB   | Ollama retains the model between requests.                     |
| Both models resident (measured peak)         | 2 592 MiB   | Recogniser contributes ≈438 MiB at <code>int8_float16</code> . |
| Headroom remaining                           | ≈1 504 MiB  | Sufficient for display, compositor, and transient allocations. |

Quantisation is what makes co-residency viable. The recogniser at `int8_float16` occupies roughly half of what half-precision would require, and the language model at Q4\_K\_M occupies approximately 2 GB against roughly 6.4 GB unquantised. Without both choices the 4 GB constraint could not be met.

## 13.3 Storage

| Artefact                       | Size    | Notes                                                                         |
|--------------------------------|---------|-------------------------------------------------------------------------------|
| Python environment             | ≈780 MB | Excluding optional CUDA wheels.                                               |
| CUDA runtime wheels (optional) | ≈1.3 GB | cuBLAS 553 MB, cuDNN 732 MB, NVRTC 76 MB. Required only for GPU acceleration. |
| Language model                 | 2.0 GB  | Managed by Ollama outside the project tree.                                   |
| Recognition model              | ≈500 MB | Cached on first use.                                                          |
| Voice model                    | ≈60 MB  | ONNX plus configuration.                                                      |
| Evaluation corpus              | ≈2.5 MB | 25 clips; excluded from version control.                                      |

## 14 Security Considerations

### 14.1 Trust Boundary

The system defines a single trust boundary: the host machine. Everything inside it — the browser, the application server, the model server, the synthesiser, and the filesystem — is treated as trusted. Nothing crosses the boundary. This is the system's principal security property and its principal simplifying assumption: the design is sound precisely as long as that boundary holds, and degrades sharply if the service is exposed beyond it.

### 14.2 Assets

| Asset                      | Sensitivity                | Exposure                                                                         |
|----------------------------|----------------------------|----------------------------------------------------------------------------------|
| Live microphone audio      | High — biometric, personal | Held in memory during a turn; never written to disk in conversational operation. |
| Recorded evaluation corpus | High — biometric           | Persisted on local disk; excluded from version control.                          |
| Transcribed text           | Moderate to high           | Persisted indefinitely in the confidence audit log.                              |
| Conversation history       | Moderate                   | Browser memory only; discarded on reload.                                        |
| Model weights              | Low                        | Publicly distributed artefacts.                                                  |

### 14.3 Controls Implemented

| ID     | Control                       | Effect                                                                                      |
|--------|-------------------------------|---------------------------------------------------------------------------------------------|
| SEC-01 | Loopback-only binding         | Services are unreachable from other hosts on the network.                                   |
| SEC-02 | No outbound network calls     | Eliminates exfiltration paths and third-party data processing entirely.                     |
| SEC-03 | Request size cap (25 MB)      | Bounds memory and disk consumption from a single request; returns HTTP 413.                 |
| SEC-04 | Structured error responses    | Prevents disclosure of stack traces, filesystem paths, and internal structure.              |
| SEC-05 | Input validation on labels    | Modulation values are checked against an allow-list before use in filenames.                |
| SEC-06 | Production WSGI server option | Avoids reliance on a development server explicitly documented as unsuitable for production. |
| SEC-07 | Version-control exclusions    | Voice recordings and logs containing transcribed speech cannot be committed accidentally.   |

| ID     | Control                      | Effect                                                                                    |
|--------|------------------------------|-------------------------------------------------------------------------------------------|
| SEC-08 | Bounded conversation context | History sent to the model is capped, limiting inadvertent propagation of earlier content. |

## 14.4 Accepted Risks

| ID     | Risk                                     | Justification and mitigation                                                                                                                                                                                           |
|--------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RSK-01 | No authentication or authorisation       | Any local process or user able to reach <code>127.0.0.1:5006</code> can use the service. Accepted for single-user local operation; documented as a hard blocker for any networked deployment (§2.3.2).                 |
| RSK-02 | No transport encryption                  | Traffic is loopback-only and does not traverse a network. HTTPS would be mandatory for any other topology, both for confidentiality and because microphone access requires a secure context.                           |
| RSK-03 | Unbounded audit log growth               | The confidence log retains transcribed speech indefinitely with no rotation or purge. Accepted at present scale; recorded as a limitation (§17) with retention guidance in §15.2.                                      |
| RSK-04 | Filename construction from request input | The modulation component is allow-listed, but the word component is not sanitised beyond trimming. Exploitation requires local access, which already implies a compromised trust boundary. Hardening is listed in §18. |
| RSK-05 | Third-party model provenance             | Model weights are consumed as published binaries and are not independently audited. This is standard practice and unavoidable at project scale, but it is a genuine supply-chain assumption.                           |
| RSK-06 | No rate limiting                         | A local caller could saturate GPU resources. Consistent with the single-user assumption.                                                                                                                               |

# 15 Ethical Considerations

This section assesses the ethical dimensions of the system as designed and as evaluated. It addresses privacy, consent, fairness, transparency, accessibility, misuse, environmental cost, licensing obligations, and the integrity of reported results. Where the system falls short, that is stated rather than omitted.

## 15.1 Privacy and Data Sovereignty

Voice is among the most sensitive categories of personal data. It is biometric — a voiceprint can identify an individual — and it carries paralinguistic signals including emotional state, and in some cases health indicators. A conversational assistant is additionally exposed to whatever the speaker says in their own home, including content concerning third parties who never consented to processing.

The system’s response is architectural rather than procedural: no audio, transcript, or generated text leaves the machine. This is not a policy commitment that could be revised, nor a setting that could be misconfigured — there is no code path to any external endpoint. The practical consequences are substantial: no third-party processor agreement is required, no cross-border transfer arises, there is no vendor-side retention to negotiate, and no account or telemetry identifier exists to correlate. Under a data-protection framework such as the GDPR, processing remains entirely within the data subject’s own control, which materially reduces — though does not eliminate — the applicable obligations.

DESIGN PRINCIPLE

Privacy here is a property of the architecture, not a feature layered on top of it. This is the strongest ethical claim the system can make, and it is also the reason the project deliberately declines to offer a hosted deployment (§2.3.2): doing so would forfeit the property that justifies the design.

## 15.2 Consent, Data Minimisation, and Retention

Two mechanisms capture voice. Conversational capture is *explicitly initiated* — the user presses a control to begin and to end recording; there is no wake word, no always-listening mode, and no ambient buffer retained beyond the ~300 ms pre-roll needed to avoid clipping speech onset. The browser additionally enforces its own permission prompt. Dataset recording is unambiguously deliberate: the evaluator is prompted per clip and reviews each recording before accepting it.

Data minimisation is largely satisfied: conversational audio is processed in memory and not persisted; conversation history exists only in browser memory and is discarded on reload. One exception is material and is stated plainly: **the confidence audit log persists transcribed speech indefinitely, in plain text, with no rotation or purge**. That log exists for a legitimate purpose — it is what makes the confidence threshold tunable from evidence and the gating behaviour auditable (§8.5.4) — but it constitutes a retention obligation that the system does not currently discharge.

| Recommendation                                                 | Rationale                                                                     |
|----------------------------------------------------------------|-------------------------------------------------------------------------------|
| Introduce rotation and a retention limit on the confidence log | Aligns the audit trail with the minimisation applied elsewhere in the system. |

| Recommendation                                                               | Rationale                                                                                                                       |
|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| Offer a redaction mode logging scores and decisions without transcribed text | Preserves threshold-calibration utility while removing content.                                                                 |
| Provide an explicit purge command                                            | Gives the data subject a practical means of erasure without manual file surgery.                                                |
| Obtain informed consent from any additional speaker recorded into the corpus | The current corpus contains one speaker who is also the developer; this ceases to be self-evident with additional contributors. |

## 15.3 Fairness and Bias

Two distinct bias surfaces exist, and both are documented rather than claimed absent.

### 15.3.1 Recognition bias

Speech recognition accuracy is well established to vary across accent, dialect, first language, age, and vocal characteristics correlated with gender. The recognition model is consumed as published; the project has not evaluated it across demographic groups and makes no claim about its equity. The reported 96% figure characterises one speaker.

### 15.3.2 Threshold calibration bias — a concrete, acknowledged defect

More specific to this system: the confidence threshold of  $-0.6$  was calibrated against a single speaker's recordings. Because the threshold governs whether a user is understood or asked to repeat, a speaker whose confidence scores are systematically lower — plausibly a speaker with a different accent, or a quieter or higher-pitched voice — would be asked to repeat more often for the same objective clarity of speech. **A fixed threshold calibrated on one voice can therefore distribute the burden of the system's uncertainty unequally.**

| Mitigation                                             | Status                |
|--------------------------------------------------------|-----------------------|
| Threshold is configurable per deployment               | Implemented           |
| Audit log provides the evidence needed to re-calibrate | Implemented           |
| Limitation documented rather than concealed            | Implemented (§17)     |
| Per-speaker adaptive calibration                       | Not implemented — §18 |
| Evaluation across multiple speakers and accents        | Not performed — §12.7 |

### 15.3.3 Language model bias

The language model reproduces biases present in its training data and may generate content that is stereotyped, culturally narrow, or factually wrong while remaining fluent. No fine-tuning, content filtering, or output moderation is applied beyond the stylistic system prompt. The observed arithmetic error during model comparison (§12.6) is a useful reminder that fluency and correctness are independent properties — a point that matters more, not less, when responses are delivered audibly and cannot be re-read.

## 15.4 Transparency and Explainability

The system makes its uncertainty visible rather than concealing it. The browser interface displays the transcription it acted upon together with its confidence score, and renders rejected transcriptions distinctly, showing the user both what was heard and why it was not trusted. The active compute device is surfaced. Every gating decision is recorded in an inspectable audit trail. This is a deliberate stance: a voice interface that silently substitutes words denies the user any means of understanding a failure, whereas one that says “I heard this, and I am not confident enough to act on it” keeps the user in control.

Limits of transparency are also acknowledged. `avg_logprob` is an uncalibrated proxy, not a probability of correctness; presenting it numerically to a non-technical user risks conveying more precision than it possesses. The language model itself is not explainable in any meaningful sense — the system can report what it heard, but not why the model answered as it did.

## 15.5 Accessibility

| Consideration               | Position                                                                                                                                                                                                                |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Alternative input path      | Text entry is available throughout, so the system remains usable by those who cannot or prefer not to speak, and in environments where speech is impractical.                                                           |
| Dual-modality output        | Every response is presented as text and as speech; neither is required to follow the conversation.                                                                                                                      |
| Atypical speech patterns    | The confidence gate degrades gracefully rather than acting on misrecognition — but users with dysarthria or atypical speech may experience elevated repeat requests, an instance of the calibration concern in §15.3.2. |
| Screen-reader compatibility | <b>Not evaluated.</b> The interface uses semantic elements and reasonable contrast, but no assistive-technology testing was performed. Stated as a gap.                                                                 |
| Timed interaction           | The browser interface uses explicit start/stop rather than a fixed recording window, avoiding a time pressure that would disadvantage slower speakers.                                                                  |

## 15.6 Misuse Potential

The system is a general-purpose local assistant with no capability to act on the world: it has no tool use, no file access on the user’s behalf, no network reach, and no persistent memory across sessions. Its misuse surface is correspondingly narrow, and the offline design removes the mass-surveillance concern that attends cloud voice assistants.

Two residual considerations are noted honestly. First, the same architecture that protects privacy also removes centralised safety oversight — an operator may substitute an unaligned model with no external control. This is an inherent property of local-first design, and the appropriate mitigation is operator responsibility rather than technical restriction. Second, the recorded voice corpus is voiceprint-grade biometric material; if such data were shared, it could contribute to voice cloning. The corpus is excluded from version control specifically to reduce the chance of inadvertent disclosure.

## 15.7 Environmental Impact

Inference draws power, and this is a genuine if modest cost. Several design decisions reduce it as a side effect of the VRAM constraint: quantised models require materially less computation than full-precision equivalents; a 3-billion-parameter model is used rather than a larger one; the recogniser loads once and is reused; and synthesis runs on CPU rather than occupying the GPU. Local inference also avoids the network transport and data-centre overhead of a hosted equivalent, though a consumer GPU is less energy-efficient per inference than optimised data-centre hardware, so the net comparison is not unambiguous. No lifecycle assessment was performed and none is claimed.

## 15.8 Licensing and Attribution

Third-party components carry obligations that would bind any redistribution of this system. These are recorded because assessing them is part of responsible engineering, not an afterthought.

TABLE 15.1 — COMPONENT LICENSING (VERIFY BEFORE ANY REDISTRIBUTION)

| Component                                    | Licence                        | Implication                                                                                                                                                                                                                                                                            |
|----------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>faster-whisper</code> ,<br>CTranslate2 | MIT                            | Permissive; attribution only.                                                                                                                                                                                                                                                          |
| Whisper model weights                        | MIT                            | Permissive.                                                                                                                                                                                                                                                                            |
| Ollama                                       | MIT                            | Permissive.                                                                                                                                                                                                                                                                            |
| Llama 3.2 3B weights                         | Llama 3.2 Community<br>Licence | <b>Not a standard open-source licence.</b> Carries acceptable-use terms and a large-scale-deployment condition. Requires review before commercial use.                                                                                                                                 |
| <code>piper-tts</code> (as installed)        | GPL-3.0-or-later               | <b>Copyleft.</b> Materially affects redistribution of any combined work. The current architecture invokes Piper as a separate subprocess rather than linking it, which is generally the less entangling arrangement, but competent legal review would be required before distribution. |
| Piper voice model                            | To be verified                 | Voice model licensing is separate from engine licensing and must be confirmed independently.                                                                                                                                                                                           |
| Flask, NumPy, SoundFile                      | BSD-family                     | Permissive.                                                                                                                                                                                                                                                                            |
| <code>librosa</code>                         | ISC                            | Permissive.                                                                                                                                                                                                                                                                            |

| Component                  | Licence                 | Implication                                                               |
|----------------------------|-------------------------|---------------------------------------------------------------------------|
| NVIDIA CUDA runtime wheels | NVIDIA proprietary EULA | Redistribution restricted; installed by the end user rather than bundled. |

#### OBLIGATION

The project is presently used privately, where these obligations are not engaged. Two items would require resolution before any public distribution: the GPL-3.0 copyleft of the synthesis engine, and the non-standard terms of the language model weights. Both are recorded here so that the decision is taken knowingly rather than by omission.

## 15.9 Integrity of Reported Results

A final ethical dimension concerns the reporting of this project's own results. The preprocessing layer was the project's stated central contribution. Measurement showed it reduced accuracy by 48 percentage points. That finding is reported here, in the README, and in the version-control history, with the full ablation and the corrective action — rather than being quietly removed, or the benchmark being re-scoped to a condition where the component appeared favourable.

This matters beyond the individual result. A benchmark constructed to confirm a hypothesis provides no information; a benchmark capable of refuting it is the only kind worth building. Reporting the refutation is what makes the remaining claims in this document credible.

## 16 Operations & Deployment

### 16.1 Installation

Dependencies are layered across three requirement files so that environments not requiring GPU acceleration are not obliged to download approximately 1.3 GB of CUDA runtime wheels.

| File                              | Adds               | Intended use                                                                  |
|-----------------------------------|--------------------|-------------------------------------------------------------------------------|
| <code>requirements.txt</code>     | Base               | Complete CPU-only operation. Sufficient to run the system.                    |
| <code>requirements-gpu.txt</code> | + cuBLAS,<br>cuDNN | GPU-accelerated recognition on the reference configuration.                   |
| <code>requirements-dev.txt</code> | + pytest, ruff     | Development and continuous integration. Deliberately excludes the GPU wheels. |

```
# 1. Create and populate the environment
python -m venv venv
venv\Scripts\python.exe -m pip install -r requirements-gpu.txt

# 2. Install the model runtime and pull the model
ollama pull llama3.2:3b

# 3. Place the Piper voice model at models/piper/
#   (en_US-lessac-medium.onnx and its .onnx.json)

# 4. Verify every dependency independently
venv\Scripts\python.exe verify_setup.py
```

`verify_setup.py` reports each of the four dependencies separately, so a single failure does not obscure the status of the others. All four report healthy on the reference configuration.

### 16.2 Configuration

No configuration is required for default operation. To override, copy `.env.example` to `.env` and edit the relevant entries; every variable is documented in Table 10.3. Values are validated at start-up, and an unparseable value raises a diagnostic naming the variable rather than failing later in an unrelated component.

### 16.3 Operating Modes

| Mode                        | Command                             | Notes                                                                                            |
|-----------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------|
| Browser interface (primary) | <code>python src/chat_web.py</code> | Open <code>http://localhost:5006</code> . Full conversational interface with multi-turn context. |
| Terminal interface          | <code>python src/cli.py</code>      | Continuous voice loop with automatic endpointing. Exit by keyword or interrupt.                  |

| Mode               | Command                                            | Notes                                                                                                                        |
|--------------------|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Dataset recorder   | <code>python src/record_web.py</code>              | Open <code>http://localhost:5005</code> . Required only when constructing an evaluation corpus.                              |
| Benchmark          | <code>python src/benchmark.py</code>               | Regenerates REF-02 and REF-03 from the current corpus.                                                                       |
| Production serving | <code>PRODUCTION=1</code> with either service      | Serves via Waitress instead of the Flask development server. Confirmed by the <code>Server: waitress</code> response header. |
| Test suite         | <code>pytest -v</code> , <code>ruff check</code> . | Requires no GPU, model server, microphone, or synthesiser.                                                                   |

## 16.4 Logging and Monitoring

| Artefact                             | Rotation             | Use                                                      |
|--------------------------------------|----------------------|----------------------------------------------------------|
| <code>logs/pipeline.log</code>       | 5 MB × 3 generations | Per-stage timings, device selection, degradation events. |
| <code>logs/confidence_log.csv</code> | None — see RSK-03    | Threshold calibration and gating audit.                  |
| <code>GET /api/health</code>         | —                    | Liveness and active recognition device.                  |

## 16.5 Troubleshooting Runbook

TABLE 16.1 — OBSERVED FAILURES AND REMEDIES

| Symptom                                                              | Cause                                                                                                        | Remedy                                                                                                                                                              |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Library <code>cublas64_12.dll</code> is not found at first inference | CUDA runtime libraries not on the process search path; construction succeeded but inference cannot (§8.4.2). | Install <code>requirements-gpu.txt</code> . <code>src/cuda_dlls.py</code> handles path injection automatically thereafter. The system falls back to CPU regardless. |
| Activation script blocked by execution policy                        | Windows PowerShell restricts script execution by default.                                                    | Invoke the interpreter directly — <code>venv\Scripts\python.exe &lt;script&gt;</code> — which requires no activation.                                               |
| Recognition device shows no value at start-up                        | Expected: the recogniser loads lazily on first use.                                                          | None. The indicator populates after the first transcription.                                                                                                        |
| First response markedly slower than subsequent ones                  | One-time model residency load in the model runtime.                                                          | None. Communicated to the user by design (NFR-P.5).                                                                                                                 |
| Diagnostic reporting the model server unreachable                    | Ollama not running, or a different address configured.                                                       | Start the Ollama service; confirm <code>OLLAMA_URL</code> .                                                                                                         |

| Symptom                                             | Cause                                                                        | Remedy                                                                                                                              |
|-----------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Frequent requests to repeat                         | Confidence threshold miscalibrated for this speaker or microphone (§15.3.2). | Inspect <code>confidence_log.csv</code> ; lower <code>CONFIDENCE_THRESHOLD</code> (more negative) if scores cluster just below it.  |
| Microphone permission not requested                 | Page not served from a secure context.                                       | Access via <code>localhost</code> , which browsers treat as secure. Any other host requires HTTPS.                                  |
| Port already in use                                 | A previous instance is still running.                                        | Terminate it, or set <code>CHAT_WEB_PORT</code> / <code>RECORD_WEB_PORT</code> .                                                    |
| Transient <code>MemoryError</code> in preprocessing | Just-in-time compilation under memory pressure (§8.3.3).                     | None required — the stage degrades to unmodified audio and logs the event. Close other memory-intensive applications if persistent. |

## 16.6 Offline Verification

The system's central claim can be verified directly by the operator: disable all network interfaces and repeat a full conversational turn. Behaviour is unchanged, because every dependency resolves to a local file or the loopback interface. This is the recommended demonstration of NFR-S.1.

## 17 Known Limitations

Limitations are recorded with their cause and, where applicable, the section that discusses them in depth. Items marked *by design* are consequences of accepted scope rather than defects.

TABLE 17.1 — KNOWN LIMITATIONS AT BASELINE

| ID     | Limitation                                                             | Class     | Detail                                                                                                                                                   |
|--------|------------------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| LIM-01 | Pitch and rate correction disabled                                     | EVIDENCED | Demonstrated harmful for isolated command words (§12.4). Detection remains active. Re-enabling requires fresh measurement and defeats a regression test. |
| LIM-02 | Confidence threshold calibrated on one speaker                         | FAIRNESS  | May distribute repeat requests unequally across speakers (§15.3.2). Configurable, with an audit trail supporting recalibration.                          |
| LIM-03 | Evaluation limited to 25 clips, one speaker, isolated words            | EVIDENCE  | Bounds the strength of accuracy claims (§12.7).                                                                                                          |
| LIM-04 | No authentication or transport encryption                              | BY DESIGN | Single-user localhost operation only (RSK-01, RSK-02).                                                                                                   |
| LIM-05 | Confidence log grows without bound and retains transcribed speech      | OPEN      | Retention obligation not discharged (RSK-03, §15.2). Remediation specified but not implemented.                                                          |
| LIM-06 | Conversation history is not persisted                                  | BY DESIGN | Browser memory only; lost on reload (§9.3). Also a privacy benefit.                                                                                      |
| LIM-07 | Synthesis begins only after generation completes                       | OPEN      | Sentence-level streaming synthesis would reduce time-to-first-audio. Identified but not implemented.                                                     |
| LIM-08 | No barge-in                                                            | BY DESIGN | The assistant cannot be interrupted mid-utterance by speech.                                                                                             |
| LIM-09 | No wake word                                                           | BY DESIGN | Explicit activation is a deliberate privacy property (§15.2).                                                                                            |
| LIM-10 | Single language (English)                                              | BY DESIGN | Recognition is pinned to English; the voice model is US English.                                                                                         |
| LIM-11 | Automated tests do not exercise real GPU, model server, or synthesiser | ACCEPTED  | Deliberate trade for CI portability; integration covered manually (§11.5).                                                                               |
| LIM-12 | No screen-reader or assistive-technology testing                       | OPEN      | Accessibility gap stated rather than assumed satisfied (§15.5).                                                                                          |
| LIM-13 | No packaging, installer, or container image                            | BY DESIGN | Distribution is by repository clone (§2.3.2).                                                                                                            |

| ID     | Limitation                                          | Class     | Detail                                                                 |
|--------|-----------------------------------------------------|-----------|------------------------------------------------------------------------|
| LIM-14 | Single concurrent user; no request queueing         | BY DESIGN | Consistent with ASM-01 and RSK-06.                                     |
| LIM-15 | Licensing obligations unresolved for redistribution | OPEN      | GPL-3.0 synthesis engine and non-standard model weight terms (\$15.8). |

## 18 Future Work

Candidate work is grouped by the objective it serves and ordered by the ratio of expected benefit to effort. Items are traceable to the limitation or risk that motivates them.

### 18.1 Near Term — closes an identified gap

| Ref   | Item                                                                 | Rationale and expected effect                                                                                                     |
|-------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| FW-01 | Rotation, retention limit, and redaction mode for the confidence log | Discharges the retention obligation in §15.2 and closes LIM-05 and RSK-03. Small, self-contained change.                          |
| FW-02 | Sentence-level streaming synthesis                                   | Begin synthesising the first complete sentence while generation continues; materially reduces time-to-first-audio. Closes LIM-07. |
| FW-03 | Filename sanitisation on recorder uploads                            | Removes RSK-04 independently of the trust-boundary argument.                                                                      |
| FW-04 | Screen-reader and keyboard-navigation audit                          | Converts LIM-12 from an unknown into a measured position.                                                                         |

### 18.2 Medium Term — strengthens the evidence base

| Ref   | Item                                              | Rationale and expected effect                                                                                                                                                             |
|-------|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FW-05 | Multi-speaker evaluation corpus                   | Addresses the most significant threat to validity (§12.7) and would quantify the fairness concern in §15.3.2 rather than leaving it as a reasoned inference.                              |
| FW-06 | Adaptive or per-speaker threshold calibration     | Derive the threshold from an initial calibration sample rather than fixing it globally. Directly mitigates LIM-02.                                                                        |
| FW-07 | Continuous-speech benchmark                       | The ablation in §12.4 justifies disabling rate correction for isolated words only. A multi-word corpus would establish whether the technique has merit in the regime it was designed for. |
| FW-08 | Noise-robustness evaluation                       | Extends validity beyond the quiet-environment assumption.                                                                                                                                 |
| FW-09 | Latency instrumentation with percentile reporting | Replaces the indicative single-run figures in §13.1 with distributions.                                                                                                                   |

### 18.3 Longer Term — extends capability

| Ref   | Item             | Rationale and expected effect                                                                                                     |
|-------|------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| FW-10 | Barge-in support | Allow the speaker to interrupt playback, which is the largest remaining gap against natural conversational rhythm. Closes LIM-08. |

| Ref   | Item                                     | Rationale and expected effect                                                                                                                                                                                 |
|-------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FW-11 | Optional persistent conversation history | Local storage with explicit user control and an erasure path. Must be weighed against the privacy benefit currently obtained from non-persistence (LIM-06).                                                   |
| FW-12 | Packaging as a distributable application | Removes the repository-clone prerequisite. Requires the licensing questions in §15.8 to be resolved first.                                                                                                    |
| FW-13 | Multilingual recognition and synthesis   | Removes LIM-10; contingent on suitable local voice models.                                                                                                                                                    |
| FW-14 | Authentication and TLS                   | Prerequisite for any deployment beyond a single local user (LIM-04). Note that this trades away the architectural privacy property described in §15.1 and should be undertaken only with that trade explicit. |

## 19 Appendices

### Appendix A — Source Inventory

| Path                                     | Category     | Purpose                                            |
|------------------------------------------|--------------|----------------------------------------------------|
| <code>src/config.py</code>               | Platform     | Validated configuration singleton.                 |
| <code>src/cuda_dlls.py</code>            | Platform     | Windows CUDA library path resolution.              |
| <code>src/browser_audio.py</code>        | Platform     | Browser media decoding to 16 kHz mono PCM.         |
| <code>src/web_common.py</code>           | Platform     | Shared error handlers; serving mode switch.        |
| <code>src/preprocessing.py</code>        | Domain       | Amplitude normalisation; pitch and rate detection. |
| <code>src/confidence_check.py</code>     | Domain       | Confidence aggregation, decision, audit trail.     |
| <code>src/pipeline.py</code>             | Domain       | Orchestration core for the conversational turn.    |
| <code>src/chat_web.py</code>             | Interface    | Conversational HTTP service.                       |
| <code>src/web_chat/index.html</code>     | Interface    | Browser conversational interface.                  |
| <code>src/record_web.py</code>           | Interface    | Dataset recorder HTTP service.                     |
| <code>src/web_recorder/index.html</code> | Interface    | Browser recording interface.                       |
| <code>src/cli.py</code>                  | Interface    | Terminal interface.                                |
| <code>src/record_test_samples.py</code>  | Evaluation   | Terminal dataset recorder.                         |
| <code>src/benchmark.py</code>            | Evaluation   | Two-condition accuracy measurement.                |
| <code>verify_setup.py</code>             | Evaluation   | Dependency verification.                           |
| <code>tests/</code>                      | Verification | 42 automated tests across four modules.            |
| <code>.github/workflows/tests.yml</code> | Verification | Continuous integration definition.                 |

### Appendix B — Key Operating Parameters

| Parameter          | Value               | Section |
|--------------------|---------------------|---------|
| Sample rate        | 16 000 Hz           | §8.2    |
| VAD frame duration | 30 ms (480 samples) | §8.2    |
| VAD aggressiveness | 2 of 0–3            | §8.2    |

| Parameter                             | Value                | Section |
|---------------------------------------|----------------------|---------|
| Trailing silence to end capture       | 800 ms               | §8.2    |
| Pre-roll retained                     | ≈300 ms (10 frames)  | §8.2    |
| Minimum speech to accept an utterance | 150 ms               | §8.2    |
| Maximum capture duration              | 15 s                 | §8.2    |
| Target RMS after normalisation        | 0.1                  | §8.3.1  |
| Pitch outlier band                    | 80–400 Hz            | §8.3.2  |
| Beam size — first pass / retry        | 1 / 5                | §8.4.3  |
| Confidence threshold                  | −0.6                 | §8.5.3  |
| Conversation history window           | 12 messages          | §8.6.2  |
| Model server timeout / retries        | 180 s / 2            | §8.6.3  |
| Maximum upload size                   | 25 MB                | §14.3   |
| Log rotation                          | 5 MB × 3 generations | §16.4   |

## Appendix C — Development History

The version-control history records the sequence in which the system was developed and, in the case of the fifth commit, preserves the evidence trail for the negative result reported in §12.

| Commit                  | Summary                                                                              | Significance                                                                               |
|-------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <a href="#">53efcd8</a> | Project scaffold                                                                     | Repository structure and exclusions.                                                       |
| <a href="#">7a0a18c</a> | Pipeline, robustness layer, benchmark, CLI                                           | Core system implementation.                                                                |
| <a href="#">427f6c8</a> | GPU enablement on Windows; TTS-aware system prompt                                   | Resolves the CUDA library defect (§8.4.2); establishes response shaping (§8.6.1).          |
| <a href="#">54e742d</a> | Browser-based dataset recorder                                                       | Enables corpus construction (UC-04).                                                       |
| <a href="#">e64b26c</a> | <b>Benchmark reveals preprocessing was reducing accuracy; fix and honest results</b> | The negative result, root-cause analysis, corrective action, and regression lock-in (§12). |
| <a href="#">ec8305a</a> | Browser conversational interface                                                     | Primary user-facing interface.                                                             |
| <a href="#">383e028</a> | Production polish: configuration, tests, CI, error handling, WSGI                    | Establishes the engineering baseline described in §11 and §16.                             |

## Appendix D — Glossary of Decisions

A consolidated index of the non-obvious decisions in this document, for reviewers seeking the reasoning rather than the specification.

| Decision                                                               | Section | One-line rationale                                                                                          |
|------------------------------------------------------------------------|---------|-------------------------------------------------------------------------------------------------------------|
| RMS rather than peak normalisation                                     | §8.3.1  | Peak level is driven by a single sample and does not represent sustained loudness.                          |
| Trial inference before trusting the GPU                                | §8.4.1  | Construction succeeds on Windows even when inference cannot; the fault is otherwise latent until first use. |
| <code>PATH</code> injection rather than <code>add_dll_directory</code> | §8.4.2  | The conventional API is silently ineffective for <code>LoadLibrary</code> - based resolution.               |
| Two-pass decoding rather than a single wide beam                       | §8.4.3  | Pays the cost of a wider search only for utterances that need it.                                           |
| Logging accepted decisions as well as rejections                       | §8.5.4  | A log of failures alone cannot establish a false-rejection rate.                                            |
| System prompt constraining response form                               | §8.6.1  | Structure that aids reading actively harms listening.                                                       |
| Retry only before first output                                         | §8.6.3  | Restarting after partial delivery would duplicate content already seen or heard.                            |
| Resolving the synthesiser from <code>sys.executable</code>             | §8.7    | Removes dependence on shell activation state.                                                               |
| Flat files rather than a database                                      | §9.3    | Evidence artefacts benefit from being directly inspectable at this scale.                                   |
| Mocking all external services in tests                                 | §11.1   | Portability and determinism, at the acknowledged cost of integration coverage.                              |
| Disabling pitch and rate correction                                    | §12.4   | Measurement demonstrated a 48-point accuracy reduction.                                                     |
| Declining hosted deployment                                            | §15.1   | It would forfeit the architectural property that justifies the design.                                      |

### END OF DOCUMENT

LVC-FTS-001 · Version 1.0 · Local Voice Chatbot — Functional & Technical Specification. Aligned with the repository at commit `383e028`. Prepared by Sandeep Srinivasan, 07 September 2026.